



Intro to Binary Exploitation

UofTCTF Seminars 2026



Overview

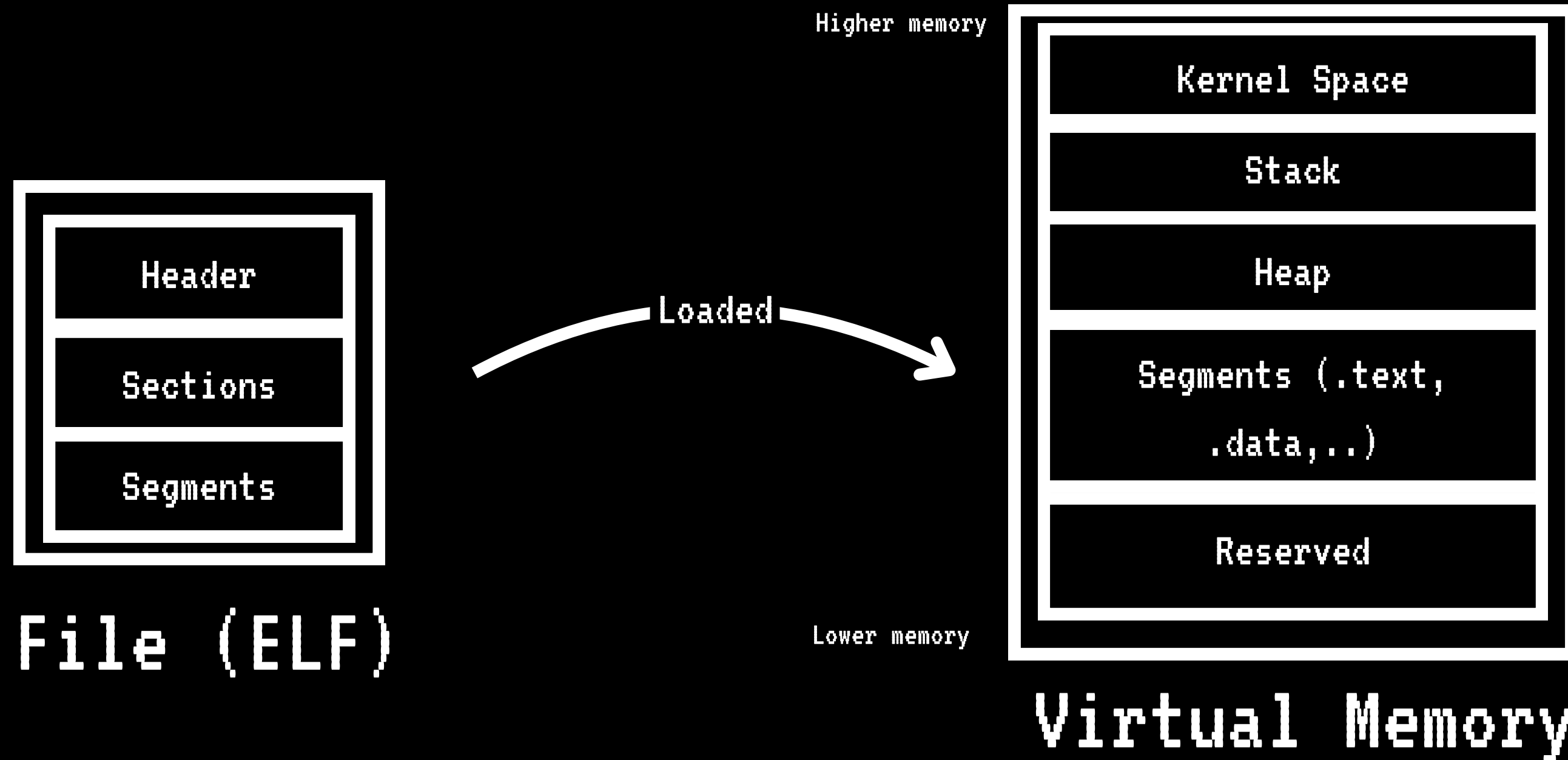
1. Prerequisite recap
2. Stack smashing
3. Shellcode
4. ROP
5. Heap
6. Misc Techniques

Announcements

We r doing pretty good on CTFTime, so we will probably be playing a ctf this weekend?

Program Memory

Programs are loaded from disk into RAM into a virtual memory space that the OS can interpret



Virtual Memory Space

This space contains:

- Binary (.text, .data, .bss, etc)
- Stack
- Heap
- Kernel code (inaccessible)
- Helper regions
- Libraries
- Memory mapped by the program



Virtual Memory

Virtual Memory Space

```
pwndbg> info proc map
process 163204
Mapped address spaces:
```

Start Addr	End Addr	Size	Offset	Perms	File
0x0000555555554000	0x0000555555555000	0x1000	0x0	r--p	/home/yoshixi/Documents/C/a.out
0x0000555555555000	0x0000555555556000	0x1000	0x1000	r-xp	/home/yoshixi/Documents/C/a.out
0x0000555555556000	0x0000555555557000	0x1000	0x2000	r--p	/home/yoshixi/Documents/C/a.out
0x0000555555557000	0x0000555555558000	0x1000	0x2000	r--p	/home/yoshixi/Documents/C/a.out
0x0000555555558000	0x0000555555559000	0x1000	0x3000	rw-p	/home/yoshixi/Documents/C/a.out
0x0000555555559000	0x0000555555557a000	0x21000	0x0	rw-p	[heap]
0x00007ffff7c00000	0x00007ffff7c24000	0x24000	0x0	r--p	/usr/lib/libc.so.6
0x00007ffff7c24000	0x00007ffff7d95000	0x171000	0x24000	r-xp	/usr/lib/libc.so.6
0x00007ffff7d95000	0x00007ffff7e04000	0x6f000	0x195000	r--p	/usr/lib/libc.so.6
0x00007ffff7e04000	0x00007ffff7e08000	0x4000	0x203000	r--p	/usr/lib/libc.so.6
0x00007ffff7e08000	0x00007ffff7e0a000	0x2000	0x207000	rw-p	/usr/lib/libc.so.6
0x00007ffff7e0a000	0x00007ffff7e12000	0x8000	0x0	rw-p	
0x00007ffff7f76000	0x00007ffff7f7b000	0x5000	0x0	rw-p	
0x00007ffff7fba000	0x00007ffff7fbe000	0x4000	0x0	r--p	[vvar]
0x00007ffff7fbe000	0x00007ffff7fc0000	0x2000	0x0	r--p	[vvar_vclock]
0x00007ffff7fc0000	0x00007ffff7fc2000	0x2000	0x0	r-xp	[vdso]
0x00007ffff7fc2000	0x00007ffff7fc3000	0x1000	0x0	r--p	/usr/lib/ld-linux-x86-64.so.2
0x00007ffff7fc3000	0x00007ffff7fed000	0x2a000	0x1000	r-xp	/usr/lib/ld-linux-x86-64.so.2
0x00007ffff7fed000	0x00007ffff7ffb000	0xe000	0x2b000	r--p	/usr/lib/ld-linux-x86-64.so.2
0x00007ffff7ffb000	0x00007ffff7ffd000	0x2000	0x39000	r--p	/usr/lib/ld-linux-x86-64.so.2
0x00007ffff7ffd000	0x00007ffff7ffe000	0x1000	0x3b000	rw-p	/usr/lib/ld-linux-x86-64.so.2
0x00007ffff7ffe000	0x00007ffff7fff000	0x1000	0x0	rw-p	
0x00007ffff7fffdd000	0x00007ffff7fffff000	0x22000	0x0	rw-p	[stack]
0xffffffffffff600000	0xffffffffffff601000	0x1000	0x0	--xp	[vsyscall]

```
munmap
```

You can view this space in GDB with `info proc map`



.text

The .text section has the executed portion of the program

Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
00001180:	F3	0F	1E	FA	F2	FF	25	FD	4D	00	00	0F	1F	44	00	00	%.M....D..
00001190:	F3	0F	1E	FA	F2	FF	25	F5	4D	00	00	0F	1F	44	00	00	%.M....D..
000011A0:	F3	0F	1E	FA	F2	FF	25	ED	4D	00	00	0F	1F	44	00	00	%.M....D..
000011B0:	F3	0F	1E	FA	F2	FF	25	E5	4D	00	00	0F	1F	44	00	00	%.M....D..
000011C0:	F3	0F	1E	FA	F2	FF	25	DD	4D	00	00	0F	1F	44	00	00	%.M....D..
000011D0:	F3	0F	1E	FA	F2	FF	25	D5	4D	00	00	0F	1F	44	00	00	%.M....D..
000011E0:	F3	0F	1E	FA	F2	FF	25	CD	4D	00	00	0F	1F	44	00	00	%.M....D..
000011F0:	F3	0F	1E	FA	F2	FF	25	C5	4D	00	00	0F	1F	44	00	00	%.M....D..
00001200:	F3	0F	1E	FA	F2	FF	25	B0	4D	00	00	0F	1F	44	00	00	%.M....D..
00001210:	F3	0F	1E	FA	F2	FF	25	B5	4D	00	00	0F	1F	44	00	00	%.M....D..
00001220:	F3	0F	1E	FA	31	ED	49	89	D1	5E	48	89	E2	48	83	E4	1.I..^H..H..
00001230:	F0	50	54	4C	8D	05	C6	15	00	00	48	8D	0D	4F	15	00	.PTL.....H..0..
00001240:	00	48	8D	3D	AF	14	00	00	FF	15	92	4D	00	00	F4	90	.H.=.....M....
00001250:	48	8D	3D	B9	4D	00	00	48	8D	05	B2	4D	00	00	48	39	H.=.M..H...M..H9
00001260:	F8	74	15	48	8B	05	6E	4D	00	00	48	85	C0	74	09	FF	.t.H..nM..H..t..
00001270:	E0	0F	1F	80	00	00	00	00	C3	0F	1F	80	00	00	00	00	
00001280:	48	8D	3D	B9	4D	00	00	48	8D	35	82	4D	00	00	48	29	H.=.M..H.5.M..H)
00001290:	FE	48	89	F0	48	C1	EE	3F	48	C1	F8	03	48	01	C6	48	.H..H..?H...H[]H
000012A0:	D1	FE	74	14	48	8B	05	45	4D	00	00	48	85	C0	74	08	.t.H..EM..H..t..
000012B0:	FF	E0	66	0F	1F	44	00	00	C3	0F	1F	80	00	00	00	00	..f..D.....
000012C0:	F3	0F	1E	FA	80	3D	6D	4D	00	00	00	75	2B	55	48	83	=mM...u+UH.
000012D0:	3D	22	4D	00	00	00	48	89	E5	74	0C	48	8B	3D	26	4D	= "M...H..t.H.=&M
000012E0:	00	00	E8	39	FE	FF	FF	E8	64	FF	FF	FF	C6	05	45	4D	...9....d....EM
000012F0:	00	00	01	5D	C3	0F	1F	00	C3	0F	1F	80	00	00	00	00	...}].....
00001300:	F3	0F	1E	FA	E9	77	FF	FF	FF	F3	0F	1E	FA	55	48	89	w.....UH.
00001310:	E5	41	55	41	54	53	48	83	EC	18	48	89	7D	D8	48	89	.AUATSH...H.}.H.
00001320:	75	D0	48	8D	3D	DF	1C	00	00	E8	22	FE	FF	FF	48	8D	u.H.=....."....H.
00001330:	0D	26	1D	00	00	48	8D	15	2D	1D	00	00	48	8D	35	33	.&...H..-...H.53
00001340:	1D	00	00	48	8D	3D	3B	1D	00	00	B8	00	00	00	00	E8	...H.=;.....
00001350:	2C	FE	FF	FF	48	8D	3D	AD	1C	00	00	E8	F0	FD	FF	FF	,...H.=.....

.plt

.text

Every byte in the .text data section is machine code

Assembly

Assembly is the mnemonic we give to machine code

48 39 c2



cmp rdx, rax

machine code (hex)

assembly

There are different flavors, we
commonly use Intel's x86 syntax

.data, .bss, .rodata

These sections are relating to global data in a program

- **.data:** literal data defined at compile time
- **.bss:** unknown data - variables with no value
- **.rodata:** read-only constants

```
00002FF0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00003000: 01 00 02 00 00 00 00 00 2B 2D 2D 2D 2D 2D 2D 2D .....+-----
00003010: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D -----
00003020: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2B 2D 2D 2D 2D 2D -----+-----
00003030: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D -----
00003040: 2D 2D 2D 2D 2B 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D ----+-----
00003050: 2D 2D 2D 2D 2D 2D 2D 2D 2D 2B 00 44 61 74 61 20 -----+ .Data
00003060: 28 4C 45 20 69 6E 74 29 00 44 61 74 61 20 28 62 (LE int).Data (b
00003070: 79 74 65 73 29 00 53 74 61 63 6B 20 6C 6F 63 61 ytes).Stack loca
00003080: 74 69 6F 6E 00 7C 20 25 33 31 73 20 7C 20 25 32 tion.| %31s | %2
00003090: 33 73 20 7C 20 25 31 38 73 20 7C 0A 00 00 00 00 3s | %18s |.....
000030A0: 7C 20 30 78 25 30 31 36 6C 78 20 28 72 73 70 2B | 0x%016lx (rsp+
000030B0: 30 78 25 30 34 78 29 20 7C 20 25 30 32 78 20 25 0x%04x) | %02x %
000030C0: 30 32 78 20 25 30 32 78 20 25 30 32 78 20 25 30 02x %02x %02x %0
000030D0: 32 78 20 25 30 32 78 20 25 30 32 78 20 25 30 32 2x %02x %02x %02
000030E0: 78 20 7C 20 30 78 25 30 31 36 6C 78 20 7C 0A 00 x | 0x%016lx |..
000030F0: 59 6F 75 20 77 69 6E 21 20 48 65 72 65 20 69 73 You win! Here is
```

.rodata

* you can see this is just raw bytes,
hex editor auto decodes to ASCII

Global Offset Table

.got:

- Stores addresses of dynamically linked functions (libc)
- Stores addresses of PLT stubs that will be resolved after first function call (lazy binding)

Function	LIBC Address
puts()	0xf7c72880
printf()	0xf7c57520
system()	0xf7c47cd0
exit()	0xf7c3a1f0

used primarily in Position Independent Executables

Syscalls

System calls are specific OS functions requested by programs:

- `read()` - read a file
- `write()` - write to file
- `time()` - get time
- `mmap()` - reserve memory
- ...

You can view the format of x86 syscalls here: <https://x86.syscall.sh>

Invoking Syscalls

We can invoke a syscall with assembly instructions.

SYSCALL NAME	references	RAX	ARG0 (rdi)	ARG1 (rsi)	ARG2 (rdx)
open	man/ cs/	2	const char *filename	int flags	umode_t mode

```
mov rbx, 0x67616c662f ; '/flag'
push rbx
mov rax, 2 ; syscall 2 - open
mov rdi, rsp ; rdi points to string '/flag'
mov rsi, 0 ; mode 0 - RDONLY
syscall
```

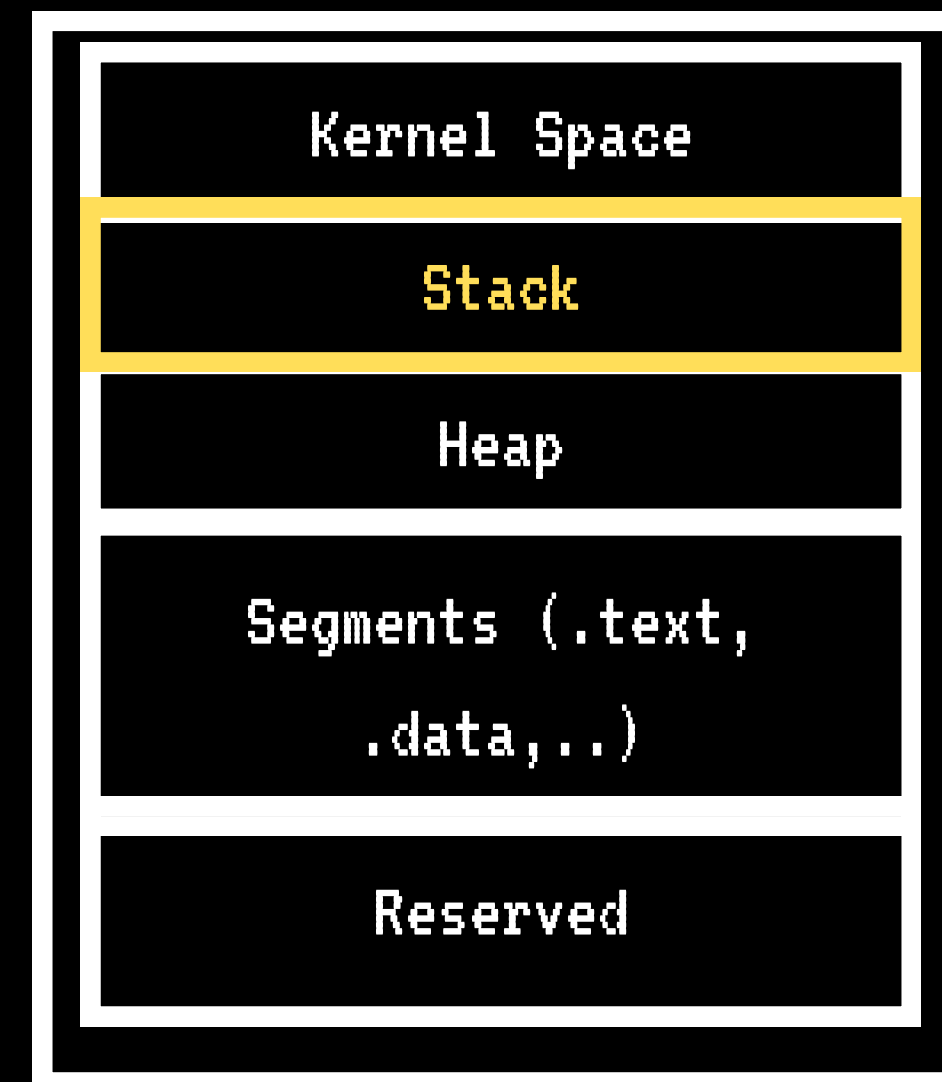
x86

You can view the format of x86 syscalls here: <https://x86.syscall.sh>

Stack

The stack is a part of virtual memory used for managing:

- Data within functions
- Functions to jump to



Virtual Memory

Stack Frame

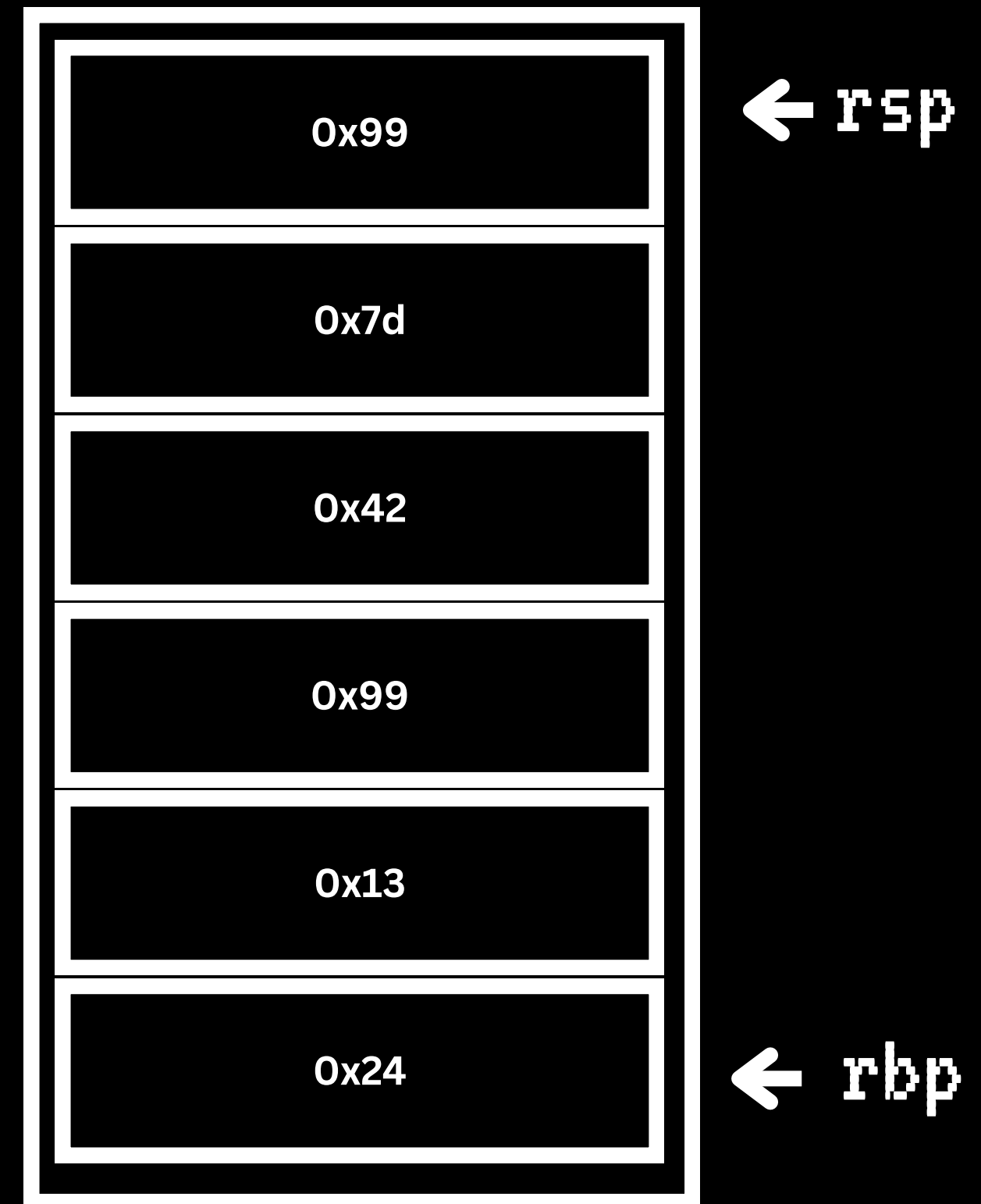
The stack frame is a section of the stack that stores info about **current function**:

- Local function variables/temporaries
- Function return info

Defined by two pointers:

rsp: stack frame top

rbp: stack frame bottom



* Stack grows down! lower addresses on top, higher address at bottom

Stack - Function Call Process

Everytime a function is called, we execute:

1. Prologue
2. Function Body
3. Epilogue

Prologue

1. `push rbp # save rbp to return to`
2. `mov rbp, rsp # update frame`
3. `sub rsp, 8 # allocate space`

Epilogue

1. `mov rsp, rbp # restore rsp`
2. `pop rbp # restore rbp`
3. `ret # return to caller`

Stack - Function Call Process

```
1 #include <stdio.h>
2 void A() {
3     int a = 10;
4     printf("In function A\n");
5 }
6 int main() {
7     A();
8     return 0;
9 }
```

: <A>:

```
55
48 89 e5
48 83 ec 10
c7 45 fc 0a 00 00 00
c6 45 fb 61
48 8d 05 b1 0e 00 00
48 89 c7
e8 d5 fe ff ff
90
c9
c3
```

```
push    rbp
mov     rbp, rsp
sub     rsp, 0x10
mov     DWORD PTR [rbp-0x4], 0xa
mov     BYTE PTR [rbp-0x5], 0x61
lea     rax, [rip+0xeb1]
mov     rdi, rax
call    1030 <puts@plt>
nop
leave
ret
```

: <main>:

```
55
48 89 e5
e8 d2 ff ff ff
b8 00 00 00 00
5d
c3
```

```
push    rbp
mov     rbp, rsp
call    1139 <A>
mov     eax, 0x0
pop     rbp
ret
```

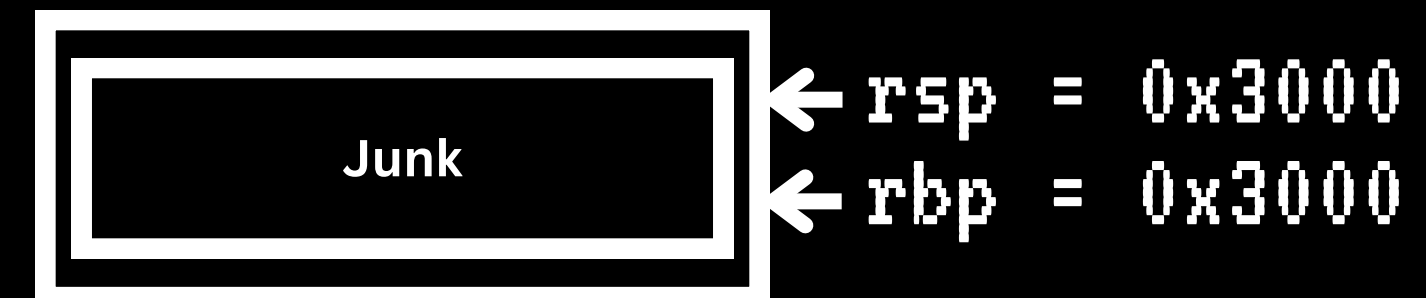
Lets view with this example

Stack - Function Call Process

<main>:

```
55
48 89 e5
e8 d6 ff ff ff
b8 00 00 00 00
5d
c3
```

```
push    rbp
mov     rbp, rsp ← rip
call    1139 <A>
mov     eax, 0x0
pop     rbp
ret
```



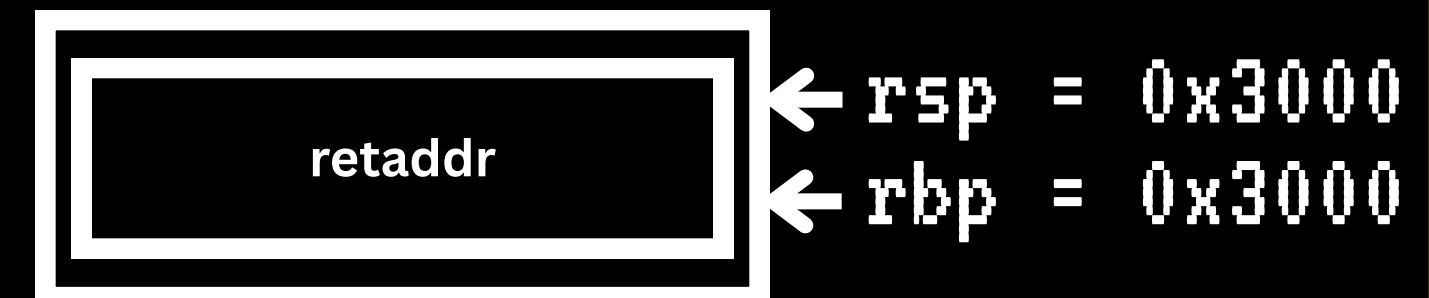
On program start, stack size is zero (rsp = rbp)

Stack - Function Call Process

<main>:

```
55
48 89 e5
e8 d6 ff ff ff
b8 00 00 00 00
5d
c3
```

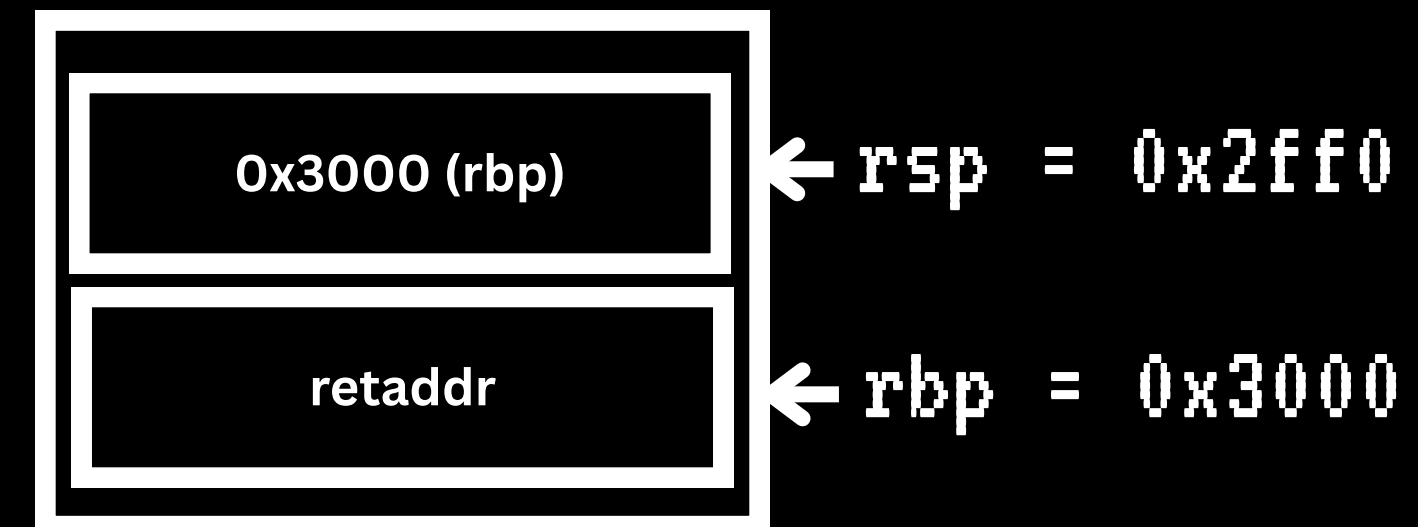
```
push    rbp
mov     rbp, rsp
call    1139 <A> ← rip
mov     eax, 0x0
pop     rbp
ret
```



call: set the current rsp to return addr, then jmp into fn

Stack - Function Call Process

```
<A>:
55      push    rbp      ← rip
48 89 e5  mov     rbp, rsp
48 83 ec 10  sub     rsp, 0x10
c7 45 fc 0a 00 00 00  mov    DWORD PTR [rbp-0x4], 0xa
c6 45 fb 61  mov     BYTE PTR [rbp-0x5], 0x61
48 8d 05 b1 0e 00 00  lea     rax, [rip+0xeb1]
48 89 c7  mov     rdi, rax
e8 d5 fe ff ff  call    1030 <puts@plt>
90      nop
c9      leave
c3      ret
```

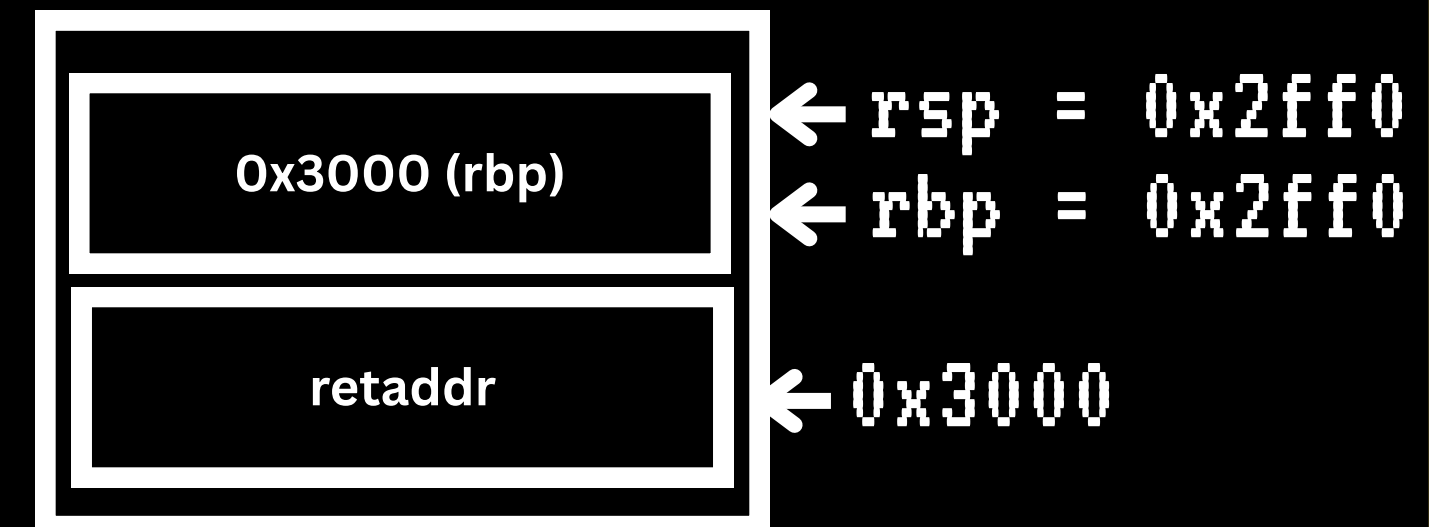


We push rbp to the top of stack

Stack - Function Call Process

<A>:

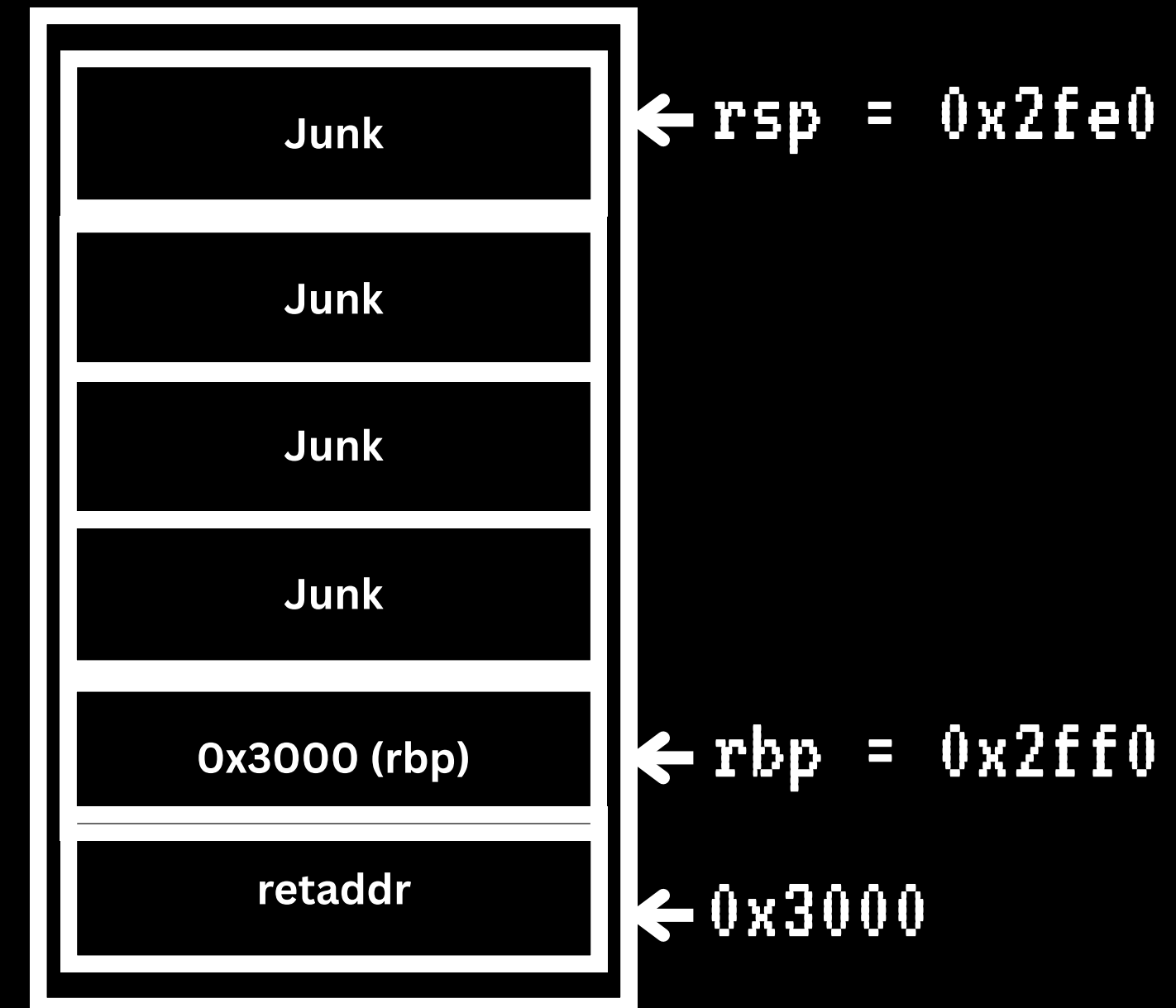
```
55      push    rbp
48 89 e5  mov     rbp, rsp    ← rip
48 83 ec 10 sub     rsp, 0x10
c7 45 fc 0a 00 00 00 mov     DWORD PTR [rbp-0x4], 0xa
c6 45 fb 61  mov     BYTE PTR [rbp-0x5], 0x61
48 8d 05 b1 0e 00 00 lea     rax, [rip+0xeb1]
48 89 c7  mov     rdi, rax
e8 d5 fe ff ff  call    1030 <puts@plt>
90      nop
c9      leave
c3      ret
```



We reset the stack frame (rsp = rbp)

Stack - Function Call Process

```
<A>:
55      push    rbp
48 89 e5  mov     rbp, rsp
48 83 ec 10 sub     rsp, 0x10 ← rip
c7 45 fc 0a 00 00 00 mov     DWORD PTR [rbp-0x4], 0xa
c6 45 fb 61  mov     BYTE PTR [rbp-0x5], 0x61
48 8d 05 b1 0e 00 00 lea     rax, [rip+0xeb1]
48 89 c7  mov     rdi, rax
e8 d5 fe ff ff  call    1030 <puts@plt>
90      nop
c9      leave
c3      ret
```

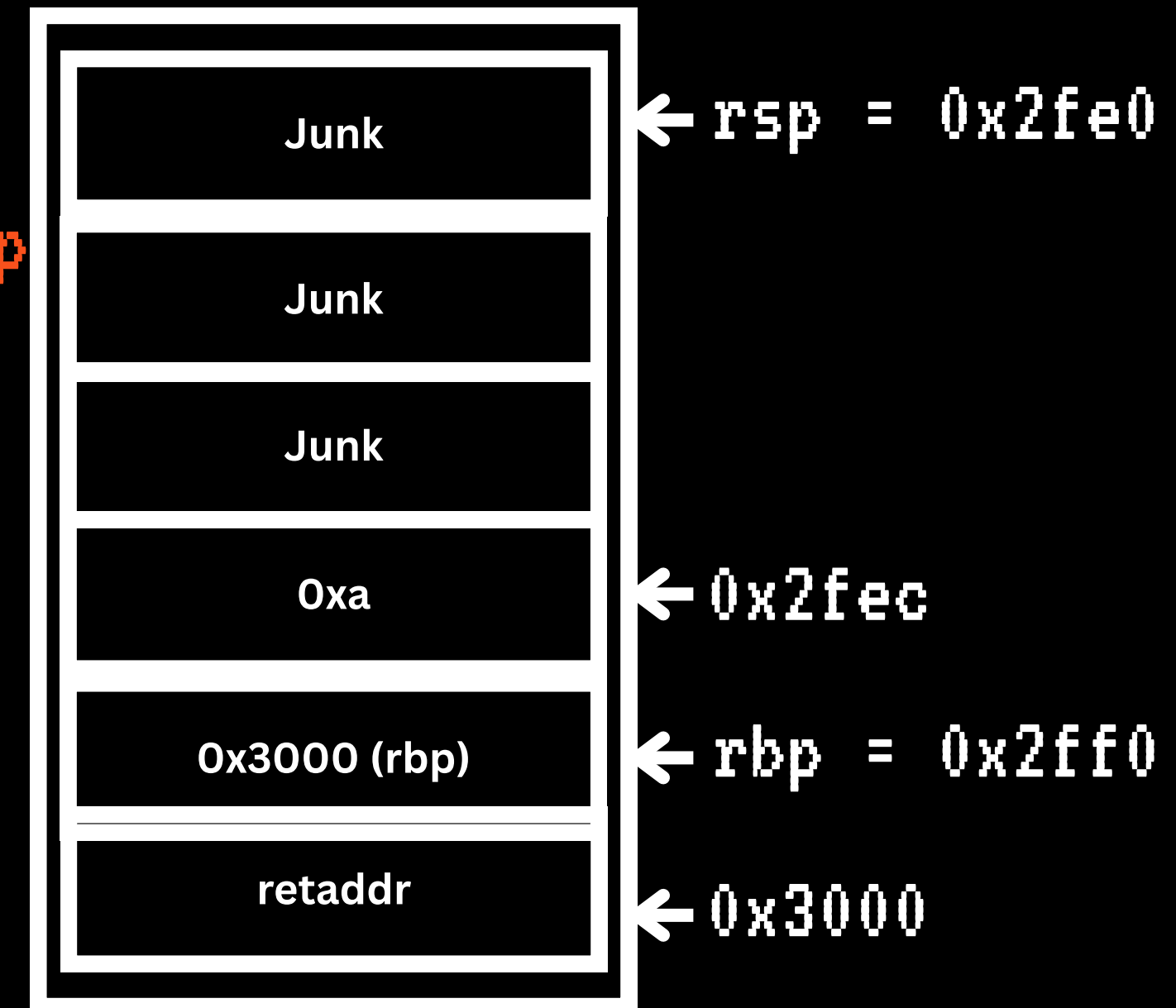


We allocate enough space (in multiples of WORDSIZE) for vars

Stack - Function Call Process

<A>:

```
55      push    rbp
48 89 e5  mov     rbp, rsp
48 83 ec 10 sub     rsp, 0x10
c7 45 fc 0a 00 00 00 mov     DWORD PTR [rbp-0x4], 0xa ← rip
c6 45 fb 61  mov     BYTE PTR [rbp-0x5], 0x61
48 8d 05 b1 0e 00 00 lea     rax, [rip+0xeb1]
48 89 c7  mov     rdi, rax
e8 d5 fe ff ff  call    1030 <puts@plt>
90      nop
c9      leave
c3      ret
```

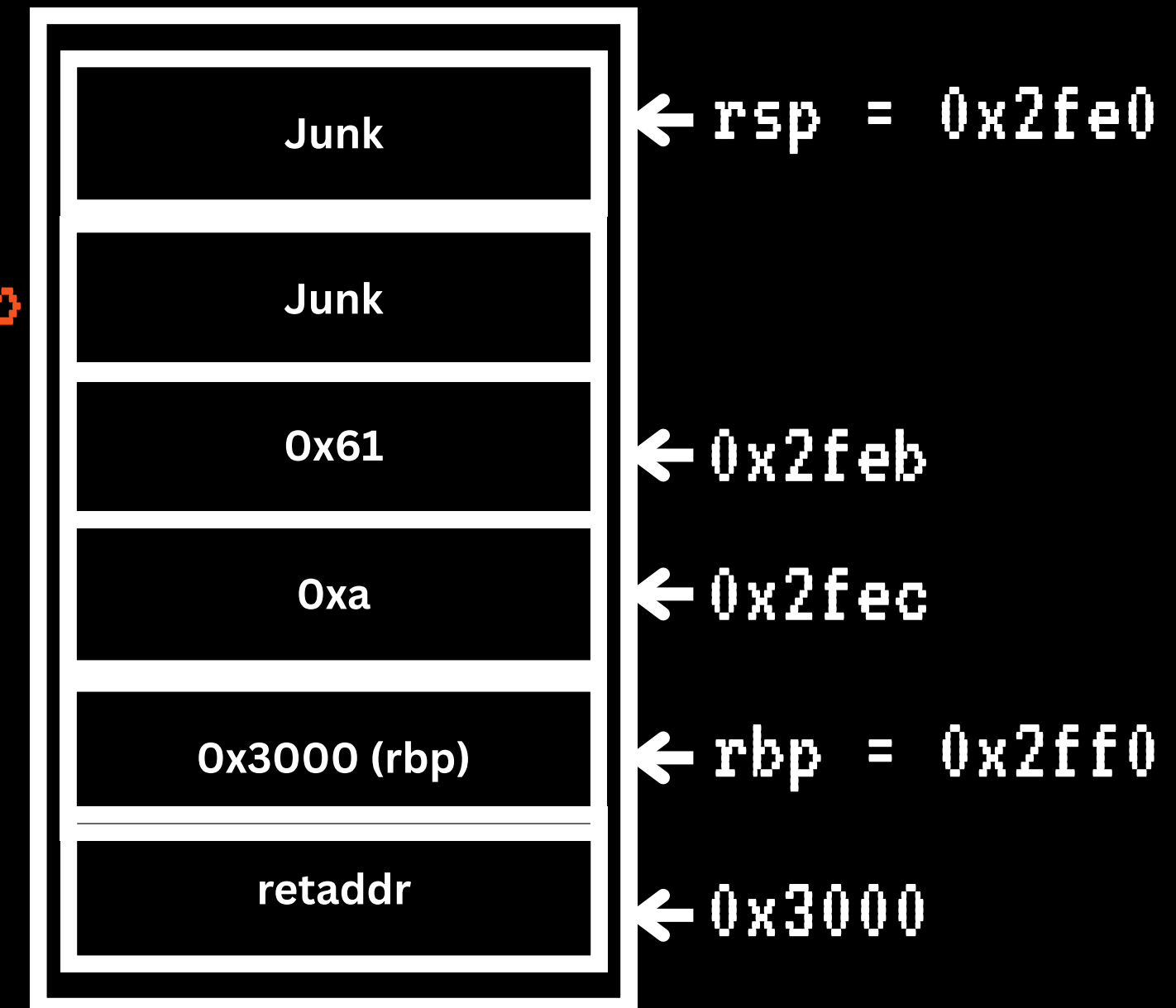


We put a integer (DWORD) onto the stack

Stack - Function Call Process

<A>:

```
55      push    rbp
48 89 e5    mov     rbp, rsp
48 83 ec 10  sub     rsp, 0x10
c7 45 fc 0a 00 00 00 mov     DWORD PTR [rbp-0x4], 0xa
c6 45 fb 61  mov     BYTE PTR [rbp-0x5], 0x61 ← rip
48 8d 05 b1 0e 00 00 lea     rax, [rip+0xeb1]
48 89 c7    mov     rdi, rax
e8 d5 fe ff ff    call    1030 <puts@plt>
90      nop
c9      leave
c3      ret
```



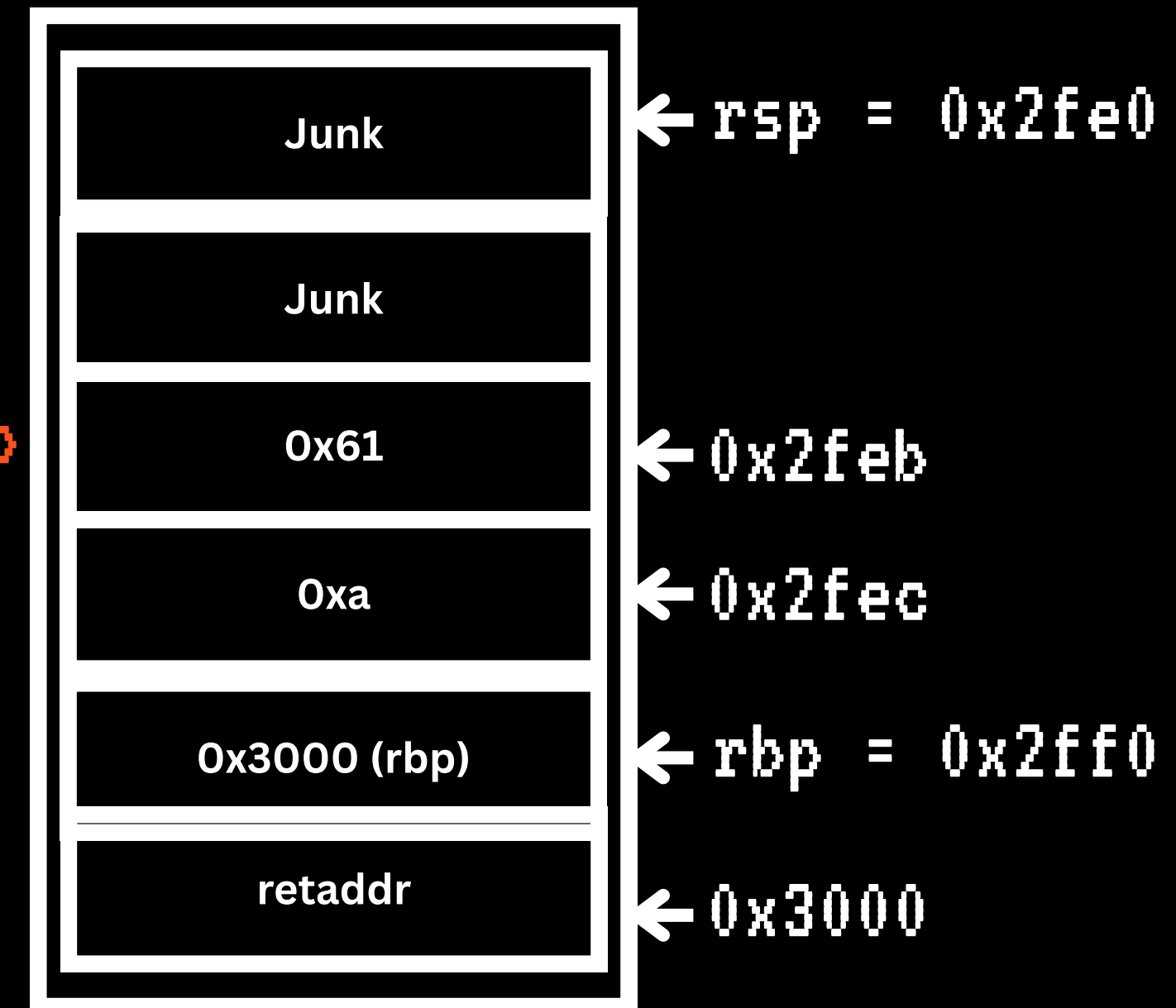
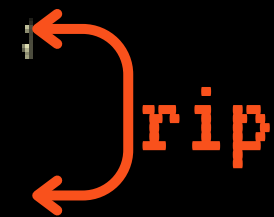
We put a integer (DWORD) onto the stack

Stack - Function Call Process

<A>:

```
55
48 89 e5
48 83 ec 10
c7 45 fc 0a 00 00 00
c6 45 fb 61
48 8d 05 b1 0e 00 00
48 89 c7
e8 d5 fe ff ff
90
c9
c3
```

```
push    rbp
mov     rbp, rsp
sub     rsp, 0x10
mov     DWORD PTR [rbp-0x4], 0xa
mov     BYTE PTR [rbp-0x5], 0x61
lea     rax, [rip+0xeb1]
mov     rdi, rax
call    1030 <puts@plt>
nop
leave
ret
```

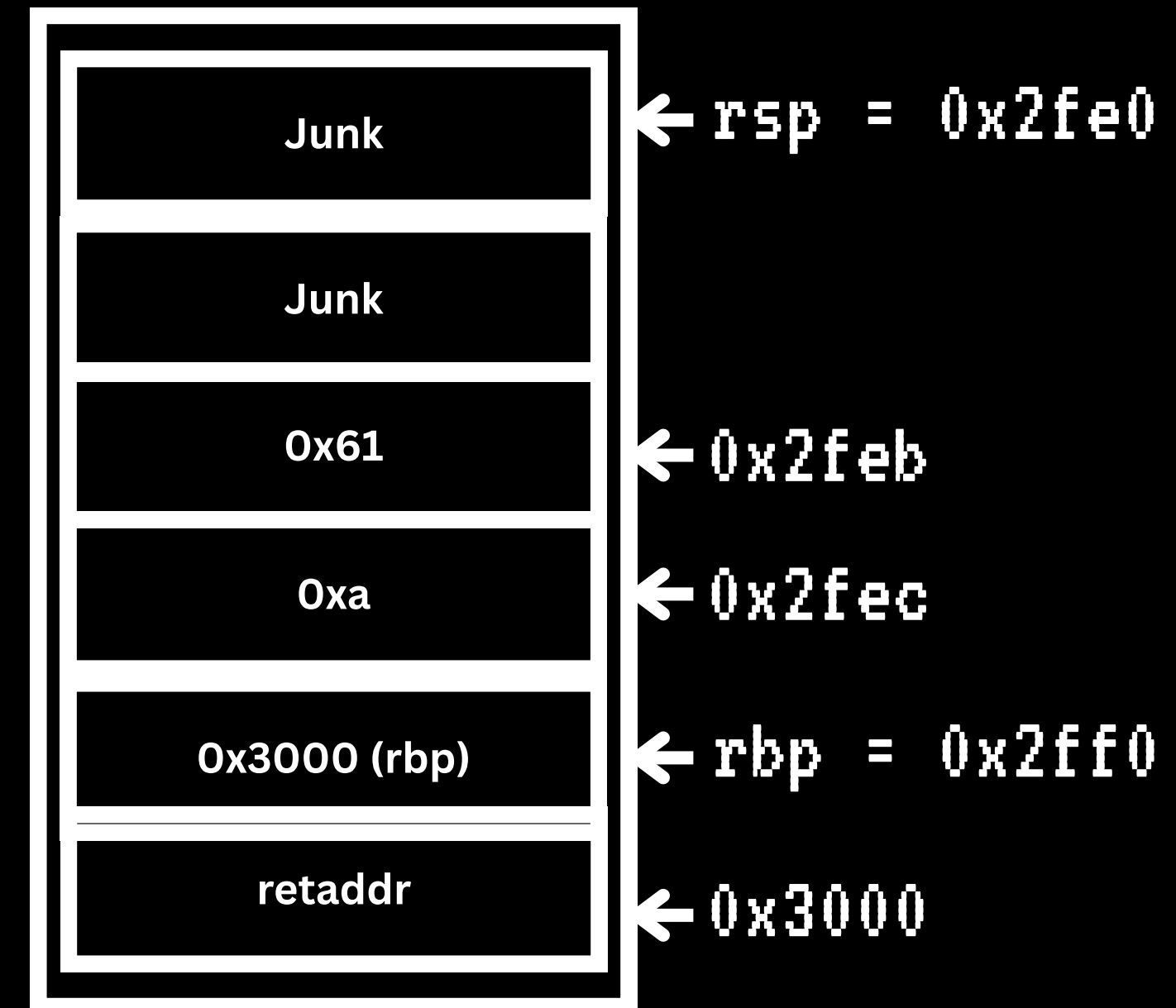


We are just doing `puts()` here

Stack - Function Call Process

<A>:

```
55      push    rbp
48 89 e5  mov     rbp, rsp
48 83 ec 10 sub     rsp, 0x10
c7 45 fc 0a 00 00 00 mov     DWORD PTR [rbp-0x4], 0xa
c6 45 fb 61  mov     BYTE PTR [rbp-0x5], 0x61
48 8d 05 b1 0e 00 00 lea     rax, [rip+0xeb1]
48 89 c7  mov     rdi, rax
e8 d5 fe ff ff  call    1030 <puts@plt>
90      nop
c9      leave   ← rip
c3      ret
```

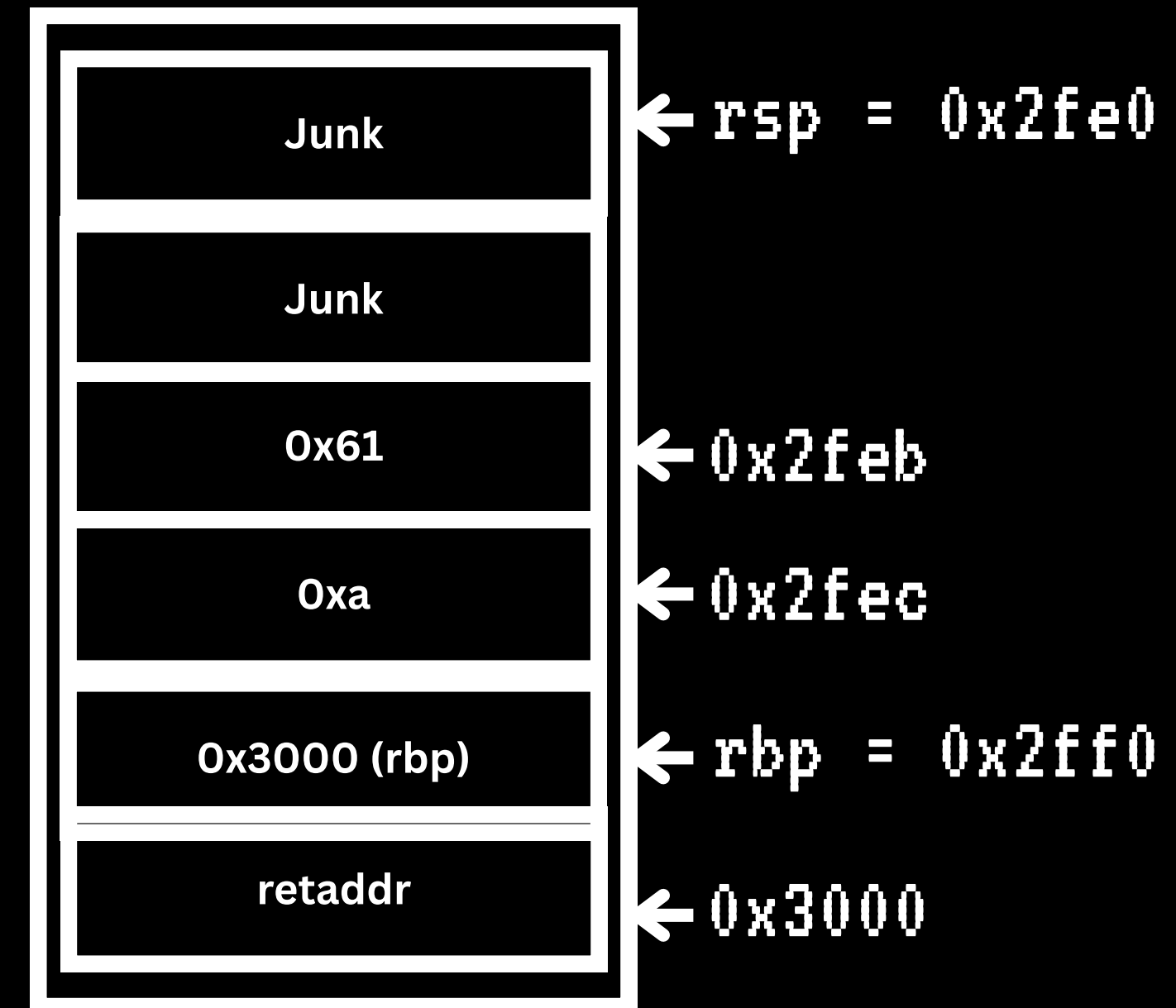


What is leave?

Stack - Function Call Process

```
<A>:
55      push    rbp
48 89 e5  mov     rbp, rsp
48 83 ec 10  sub     rsp, 0x10
c7 45 fc 0a 00 00 00  mov     DWORD PTR [rbp-0x4], 0xa
c6 45 fb 61  mov     BYTE PTR [rbp-0x5], 0x61
48 8d 05 b1 0e 00 00  lea     rax, [rip+0xeb1]
48 89 c7  mov     rdi, rax
e8 d5 fe ff ff  call    1030 <puts@plt>
90      nop
c9      leave   ← rip
c3      ret
```

```
48 89 e5  mov     rbp, rsp
5d      pop     rbp
```



leave is a macro for these two instructions

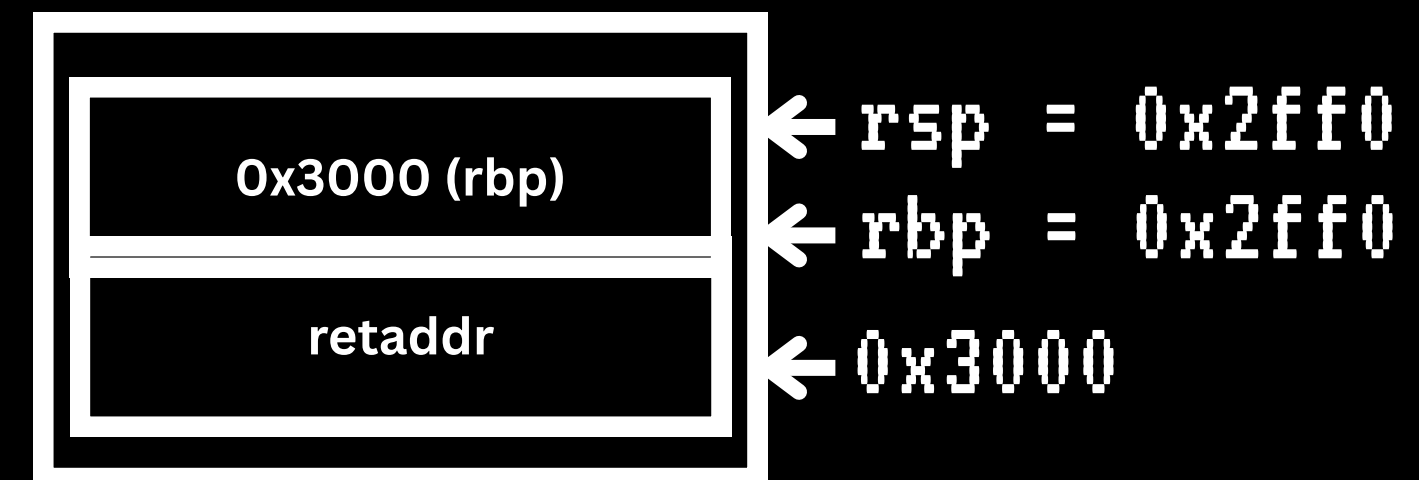
Stack - Function Call Process

<A>:

```
55      push    rbp
48 89 e5  mov     rbp, rsp
48 83 ec 10 sub     rsp, 0x10
c7 45 fc 0a 00 00 00 mov     DWORD PTR [rbp-0x4], 0xa
c6 45 fb 61  mov     BYTE PTR [rbp-0x5], 0x61
48 8d 05 b1 0e 00 00 lea     rax, [rip+0xeb1]
48 89 c7  mov     rdi, rax
e8 d5 fe ff ff  call    1030 <puts@plt>
90      nop
c9      leave
c3      ret
```

```
48 89 e5
5d
```

```
mov     rbp, rsp ← rip
pop     rbp
```



delete the stack frame

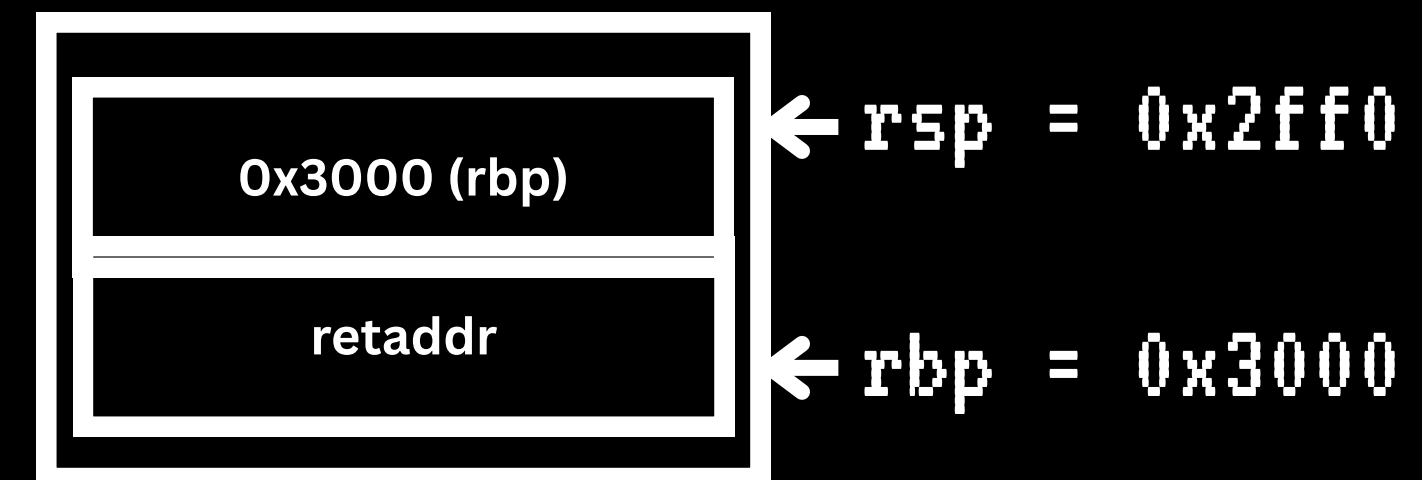
Stack - Function Call Process

<A>:

```
55      push    rbp
48 89 e5  mov     rbp, rsp
48 83 ec 10 sub     rsp, 0x10
c7 45 fc 0a 00 00 00 mov     DWORD PTR [rbp-0x4], 0xa
c6 45 fb 61  mov     BYTE PTR [rbp-0x5], 0x61
48 8d 05 b1 0e 00 00 lea     rax, [rip+0xeb1]
48 89 c7  mov     rdi, rax
e8 d5 fe ff ff  call    1030 <puts@plt>
90      nop
c9      leave
c3      ret
```

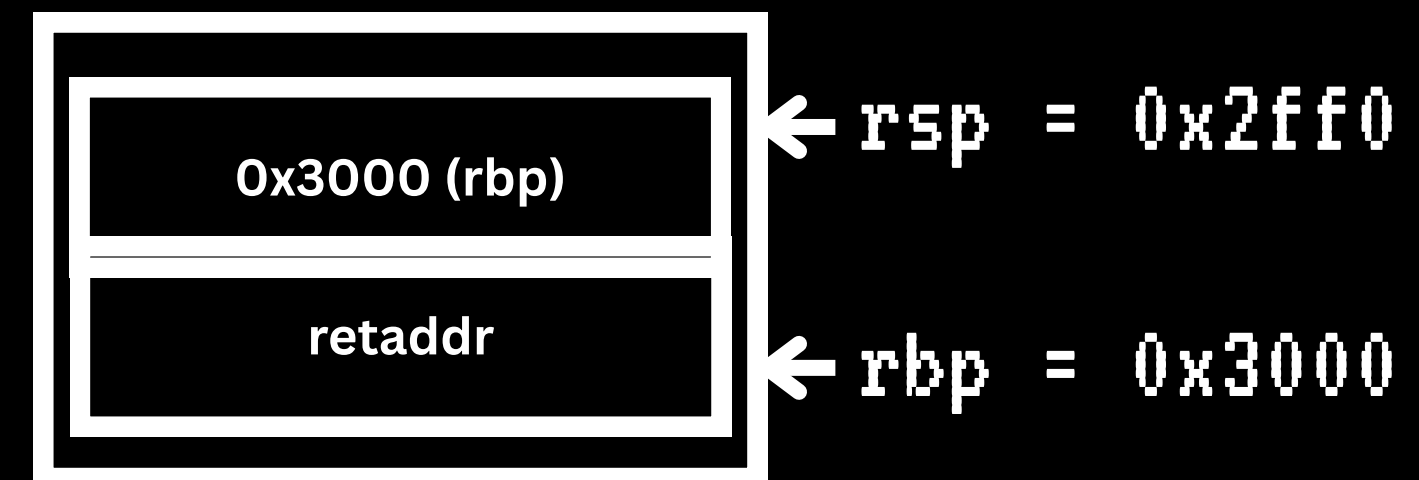
```
48 89 e5  mov     rbp, rsp
5d      pop     rbp
```

restore rbp



Stack - Function Call Process

```
<A>:
55      push    rbp
48 89 e5  mov     rbp, rsp
48 83 ec 10 sub     rsp, 0x10
c7 45 fc 0a 00 00 00 mov     DWORD PTR [rbp-0x4], 0xa
c6 45 fb 61  mov     BYTE PTR [rbp-0x5], 0x61
48 8d 05 b1 0e 00 00 lea     rax, [rip+0xeb1]
48 89 c7  mov     rdi, rax
e8 d5 fe ff ff  call    1030 <puts@plt>
90      nop
c9      leave
c3      ret     ← rip
```



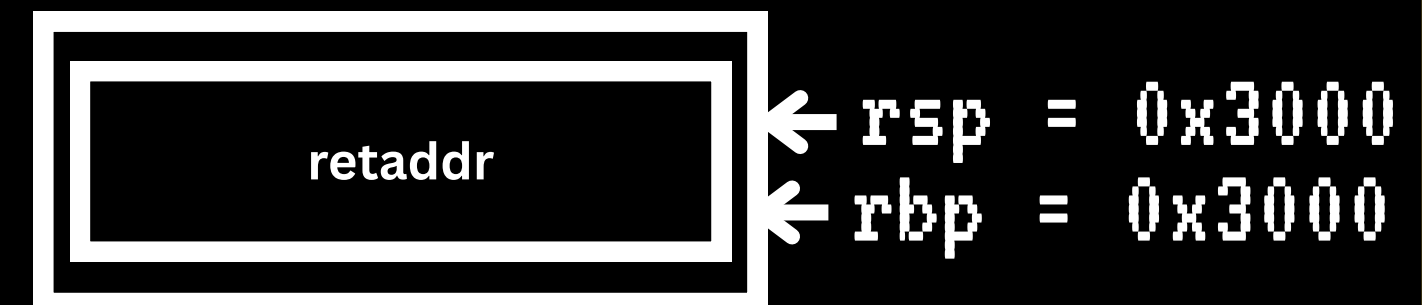
return: set rip to value at rbp, add rsp 8

Stack - Function Call Process

<main>:

```
55
48 89 e5
e8 d6 ff ff ff
b8 00 00 00 00
5d
c3
```

```
push    rbp
mov     rbp, rsp
call    1139 <A>
mov     eax, 0x0 ← rip
pop     rbp
ret
```



Ok we are done!

Calling Conventions - cdecl

For Linux 32-bit binaries

Function arguments are passed through stack in **reverse** order.

Source

```
int myfunc(int a,
           int b,
           int c){
    return a*b-c;
}
int main(){
    int val = myfunc(4,5,6);
    return 0;
}
```

Segment of <main>

```
sub    esp,0x10
call   11ae <__x86.get_pc_thunk.ax>
add    eax,0x2e63
push   0x6
push   0x5
push   0x4
call   116d <myfunc>
```

Segment of <myfunc>

```
add    eax,0x2e7f
mov     eax,DWORD PTR [ebp+0x8]
imul    eax,DWORD PTR [ebp+0xc]
sub     eax,DWORD PTR [ebp+0x10]
pop     ebp
ret
```

Args passed in reverse

Inside fn, we directly
access args from stack

Calling Conventions - SysV

For Linux 64-bit binaries

Function arguments are passed through:

Arg1: rdi

Arg2: rsi

Arg3: rdx

Arg4: rcx

Arg5: r8

Arg6: r6

Extra arguments pushed onto the stack (like cdecl)

Calling Conventions - cdecl

For Linux 64-bit binaries

Source

```
int myfunc(int a,
           int b,
           int c){
    return a*b-c;
}
int main(){
    int val = myfunc(4,5,6);
    return 0;
}
```

Segment of <main>

```
push    rbp
mov     rbp, rsp
sub     rsp, 0x10
mov     edx, 0x6
mov     esi, 0x5
mov     edi, 0x4
call    1119 <myfunc>
mov     DWORD PTR [rbp-0x4], eax
mov     eax, 0x0
leave
ret
```

Args directly in registers

Segment of <myfunc>

```
push    rbp
mov     rbp, rsp
mov     DWORD PTR [rbp-0x4], edi
mov     DWORD PTR [rbp-0x8], esi
mov     DWORD PTR [rbp-0xc], edx
mov     eax, DWORD PTR [rbp-0x4]
imul    eax, DWORD PTR [rbp-0x8]
sub     eax, DWORD PTR [rbp-0xc]
pop     rbp
ret
```

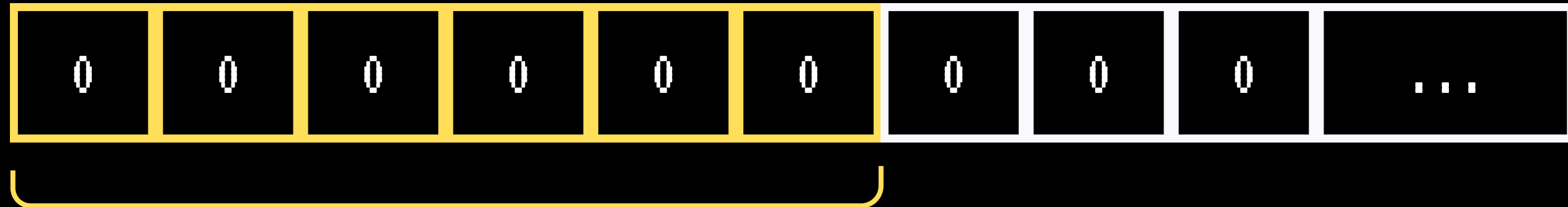
Move register values onto the
stack as local temporaries

Stack Smashing

Buffer Overflow

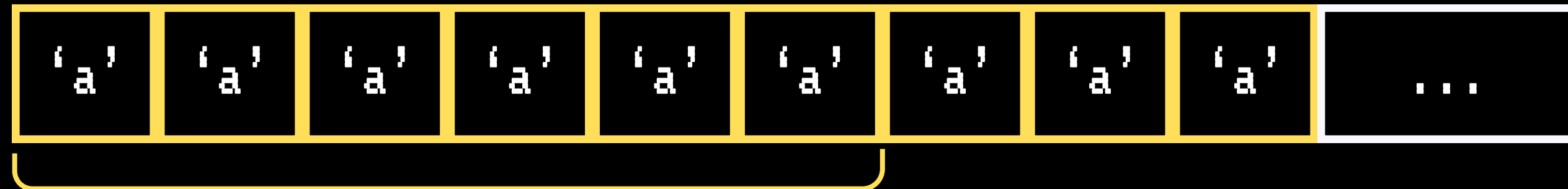
A input buffer can be overflowed if input data exceeds intended size

Reserved:



input buffer

After input:

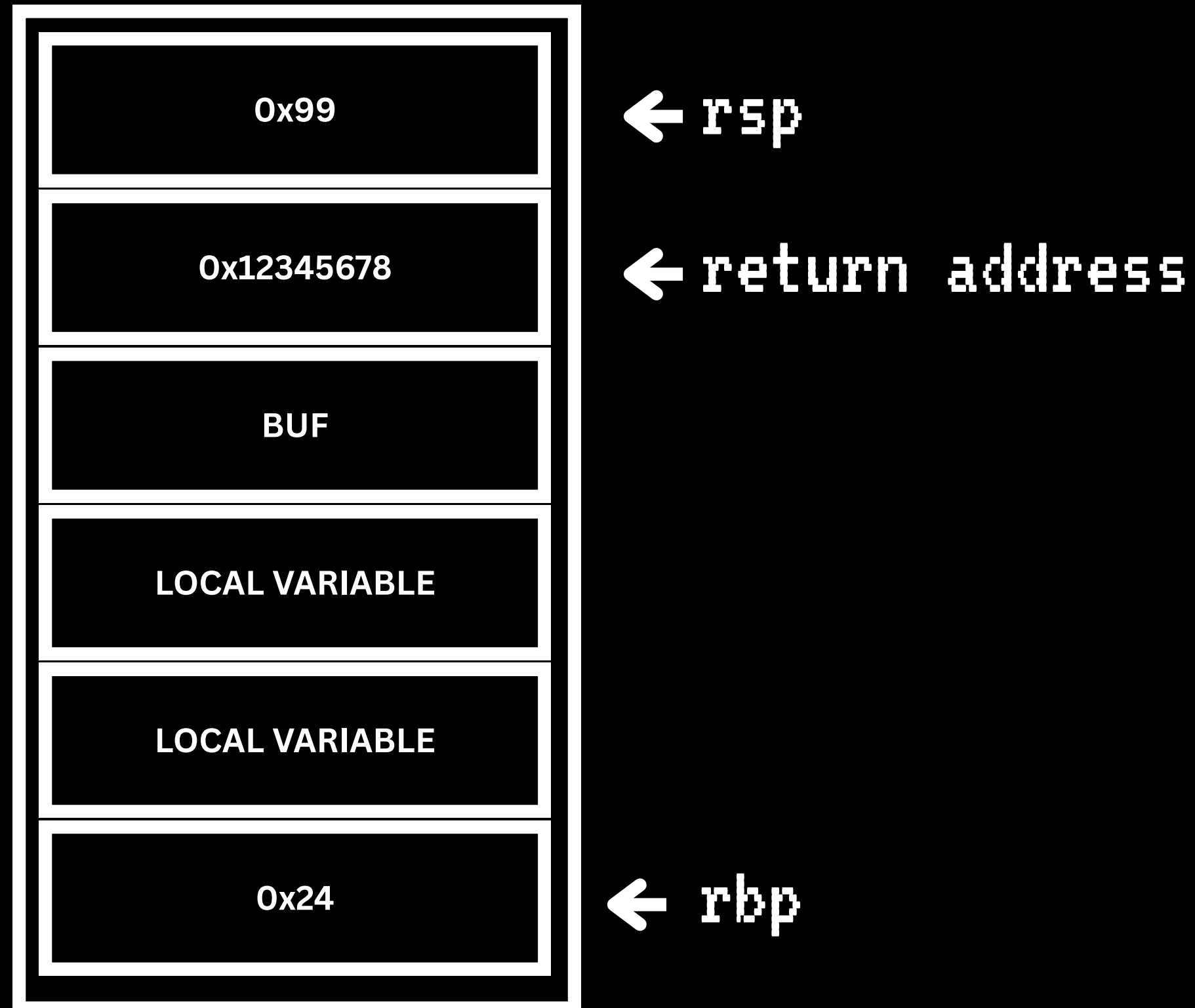


input buffer

Memory can be overflowed on the:

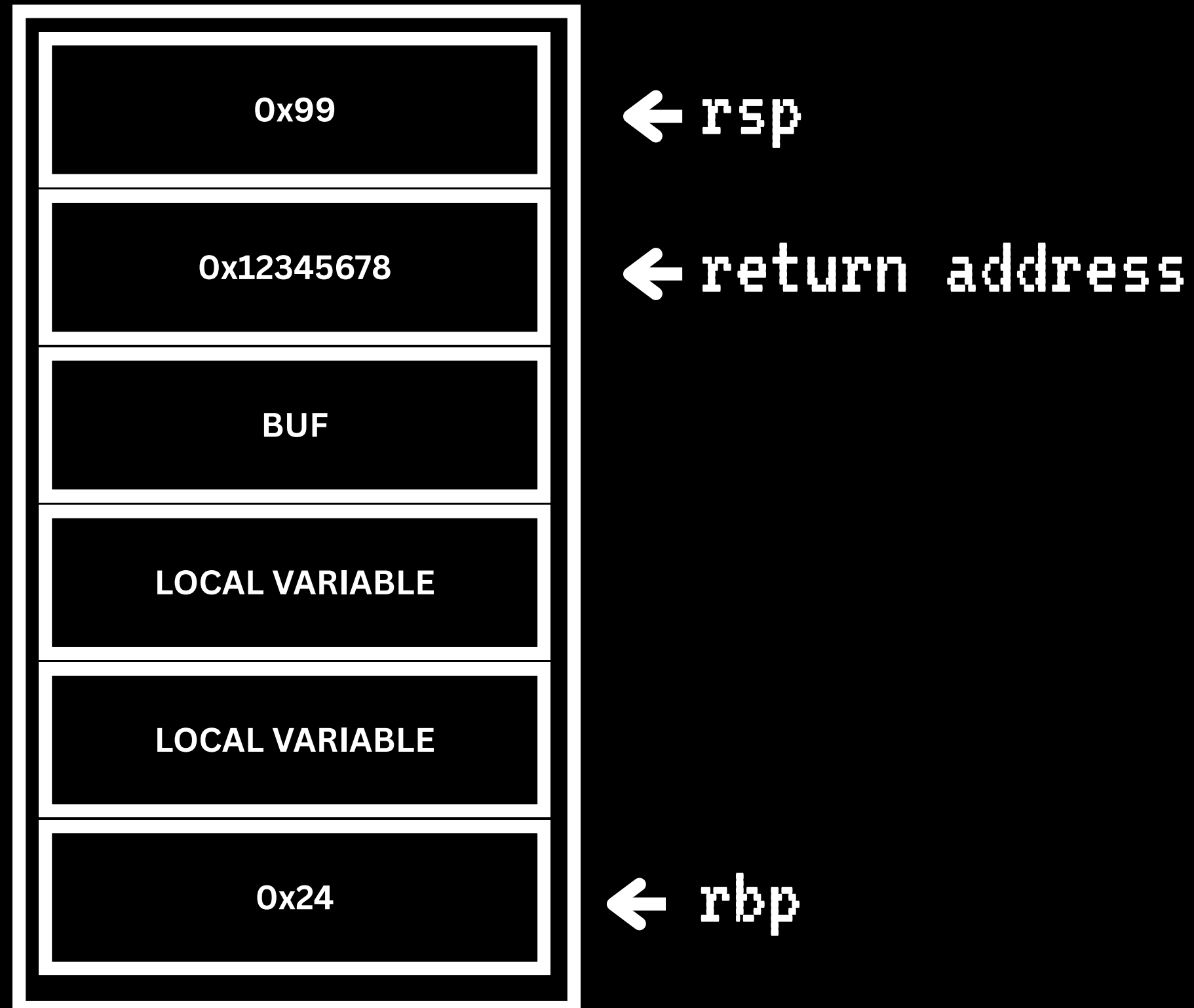
- Stack
- Heap

Stack Overflow/Smashing



A overflow can occur
on the stack if there
are no input size
checks

Stack Overflow/Smashing



Let's spam a bunch of
A's into the buffer
:smile:

Stack Overflow/Smashing



← rsp

← return address

← rbp

Oh no, it's over for
us, now it will try to
return to 0x41414141

Stack Overflow/Smashing



← rsp

← return address

← rbp

We can change the
return address to
somewhere we control

Stack Overflow/Smashing

Lets try and exploit this piece of code:

```
1 #include <stdio.h>
2 void win(){
3     printf("i have been pwned");
4 }
5
6 int main(){
7     char buff[100];
8     printf("write your post here: ");
9     gets(buff);
10    printf("you entered: %s", buff);
11    return 0;
12 }
```

Follow along with files here:

<https://github.com/UofTCTF/intro-binexp-workshop-challenges>



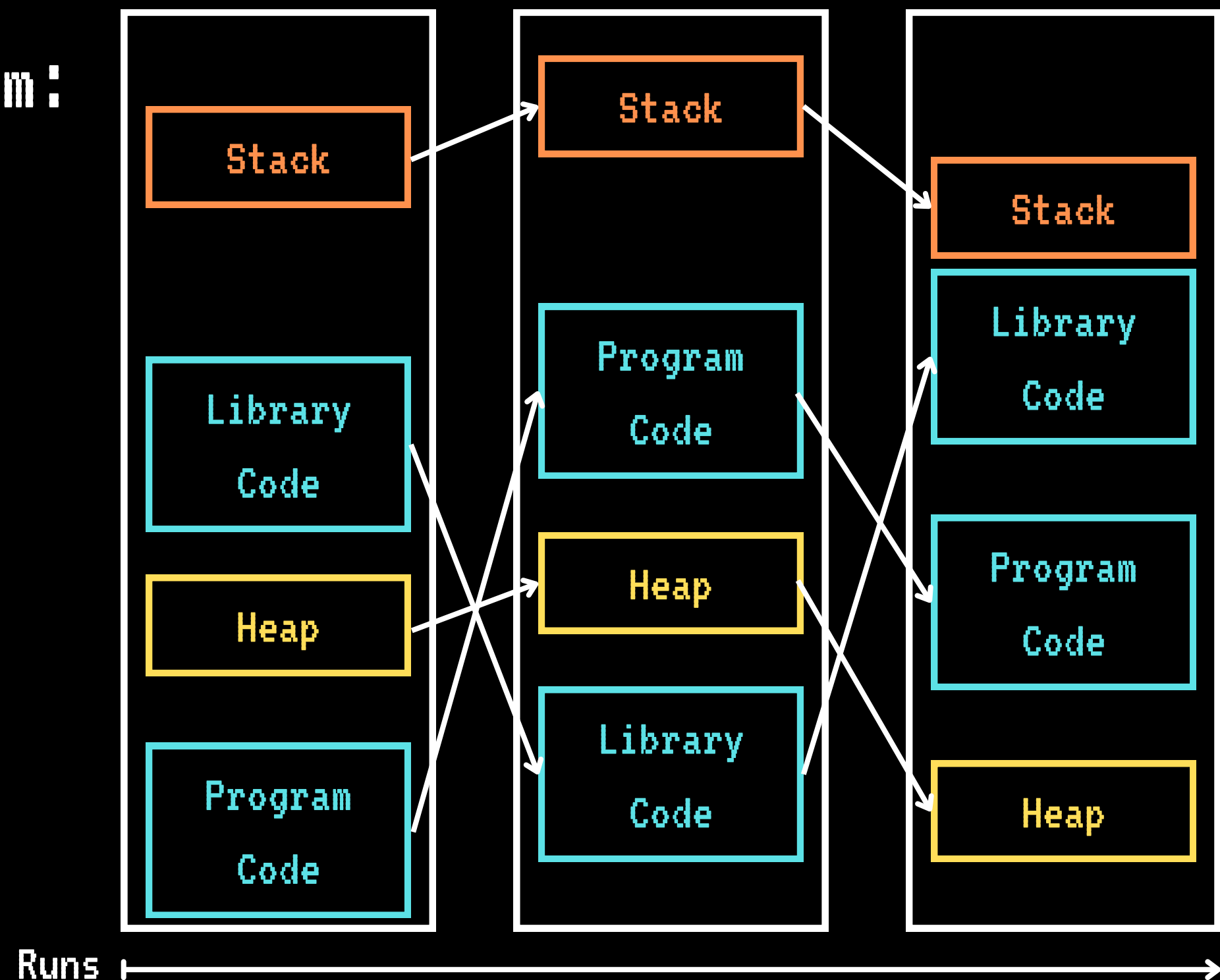
Mitigation - ASLR

ASLR: Address Space Layout Randomization.

Every program run has a random:

- Base address
- Address of stack
- Address of heap
- Address of libraries

Prevents us from knowing mapped addresses beforehand



Mitigation - ASLR

ASLR can be disabled locally if you have root perms:

In gdb:

```
set disable-randomization on
```

Making a new shell with ASLR disabled:

```
setarch x86_64 -R /bin/bash
```

ASLR Workarounds

You can work around ASLR by:

- Leaking the addresses you require using another vulnerability (format string, ROP, etc)
- Brute forcing address on forked processes
- Brute forcing page offsets

Position Independent Executable (PIE)

A requirement for ASLR - The program can execute the same way regardless of the address it is loaded at.

```
#include <stdio.h>
char mystring[] = "hello";
int main(){
    printf("%s", mystring);
    return 0;
}
```

```
lea    rdx, [rip + 0x2ed4]
lea    rax, [rip + 0xeb9]
mov     rsi, rdx
mov     rdi, rax
mov     eax, 0
call   printf@plt
```

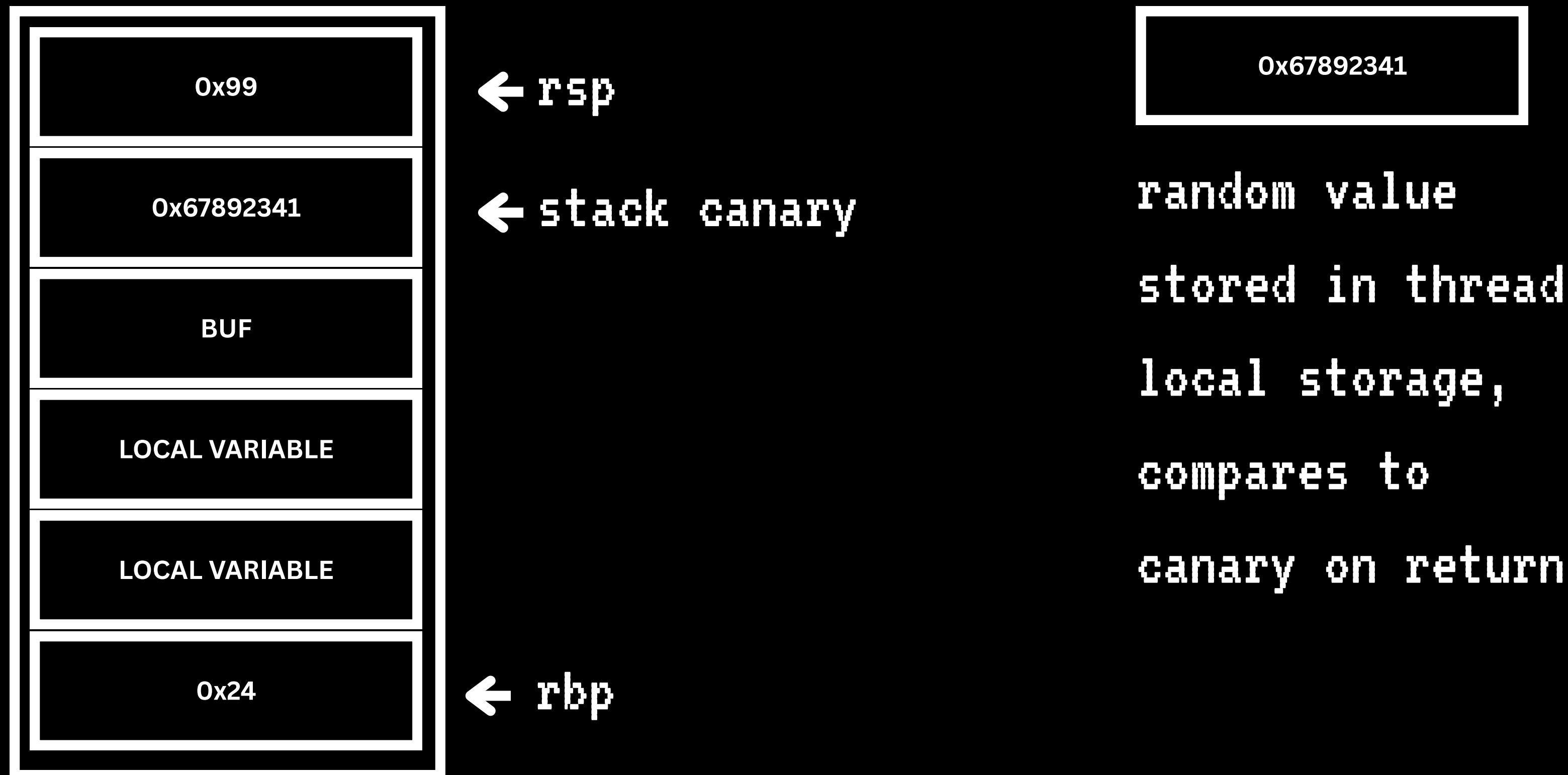
rip ->



Data in other sections referenced

relative to the instruction pointer **rip**

Mitigation - Stack Canary



Mitigation - Stack Canary

w/o Canary

```
push    rbp
mov     rbp, rsp
sub     rsp, 0x70
lea     rax, [rbp-0x70]
mov     rdi, rax
call    401030 <gets@plt>
mov     eax, 0x0
leave
ret
```

Source

```
int main(){
    char buff[100];
    gets(buff);
    return 0;
}
```

with Canary

```
push    rbp
mov     rbp, rsp
sub     rsp, 0x70
mov     rax, QWORD PTR fs:0x28           generate random
                                           canary from
                                           thread storage
mov     QWORD PTR [rbp-0x8], rax
xor     eax, eax
lea     rax, [rbp-0x70]
mov     rdi, rax
call    401040 <gets@plt>
mov     eax, 0x0
mov     rdx, QWORD PTR [rbp-0x8]         compare stack
sub     rdx, QWORD PTR fs:0x28           bottom with canary

je      401172 <main+0x3c>                crash if
call    401030 <__stack_chk_fail@plt>    not same
leave
ret
```

Bypass - Stack Canary

Stack Canary can be bypassed if:

- You leak the stack canary with another vulnerability
- Overwrite the stack non-sequentially

Shellcode

Shellcode

This is custom-made machine code designed for exploitation purposes

```
.intel_syntax noprefix
.global _start
_start:
mov rax, 59
lea rdi, [rip+binsh]
mov rsi, 0
mov rdx, 0
syscall
binsh:
.string "/bin/sh"
```



Assembler



```
48c7 c03b 0000 0048 8d3d 1000 0000
48c7 c600 0000 0048 c7c2 0000 0000
0f05 2f62 696e 2f73 6800
```

Shellcode

ASM to spawn shell

Vonn Neumann Arcitecture

A computer architecture where all sections of program memory are:

- Executable
- Writable

This allows us to think of **data = instructions**

Shellcode Injection

If we are given an input and the ability to change control flow of program, we can execute any assembly code we want.

1. Feed shellcode instructions into the first input
2. Change the return address to point to the stack address of where shellcode was stored
3. Program returns and set rip to that address and runs the shellcode

Creating Shellcode

1. Write your assembly in a file shellcode.s
2. Assembly and link to create a object file

```
gcc -Wl,-N -nostdlib -static shellcode.s -o shellcode-elf
```

3. Extract the .text section from the elf

```
objcopy --dump-section .text=shellcode-raw shellcode-elf
```

Use python to send the shellcode-raw into a program

Shellcode Injection

Heres an example challenge:

```
#include <stdlib.h>
#include <stdio.h>
int main() {
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    char buff[100];
    printf("write your post here: \n");
    printf("buff pointer: %p", buff);
    gets(buff);
    printf("your post: %s\n", buff);
    return 0;
}
```

Follow along with files here:

<https://github.com/UofTCTF/intro-binexp-workshop-challenges>

Pwntools Shellcraft

Python library pwntools has a shellcraft feature which can make shellcode for common tasks:

```
In [1]: import pwn
```

```
In [2]: payload = pwn.shellcraft.amd64.linux.cat("/flag") # shellcode for catting flag
```

```
In [3]: payload2 = pwn.shellcraft.amd64.linux.sh()
```

```
In [4]: print(payload)
/* push b'/flag\x00' */
mov rax, 0x101010101010101
push rax
mov rax, 0x101010101010101 ^ 0x67616c662f
xor [rsp], rax
/* call open('rsp', 'O_RDONLY', 'rdx') */
push SYS_open /* 2 */
pop rax
mov rdi, rsp
xor esi, esi /* O_RDONLY */
syscall
```

<https://docs.pwntools.com/en/stable/shellcraft/amd64.html>

Mitigation - DEP/NX



DEP: Data Execution Prevent (Windows)

NX: No-eXecute (Linux)

Introduces a NX/W^X bit for setting memory permissions.

W^X: Memory is either writable or executable but not both

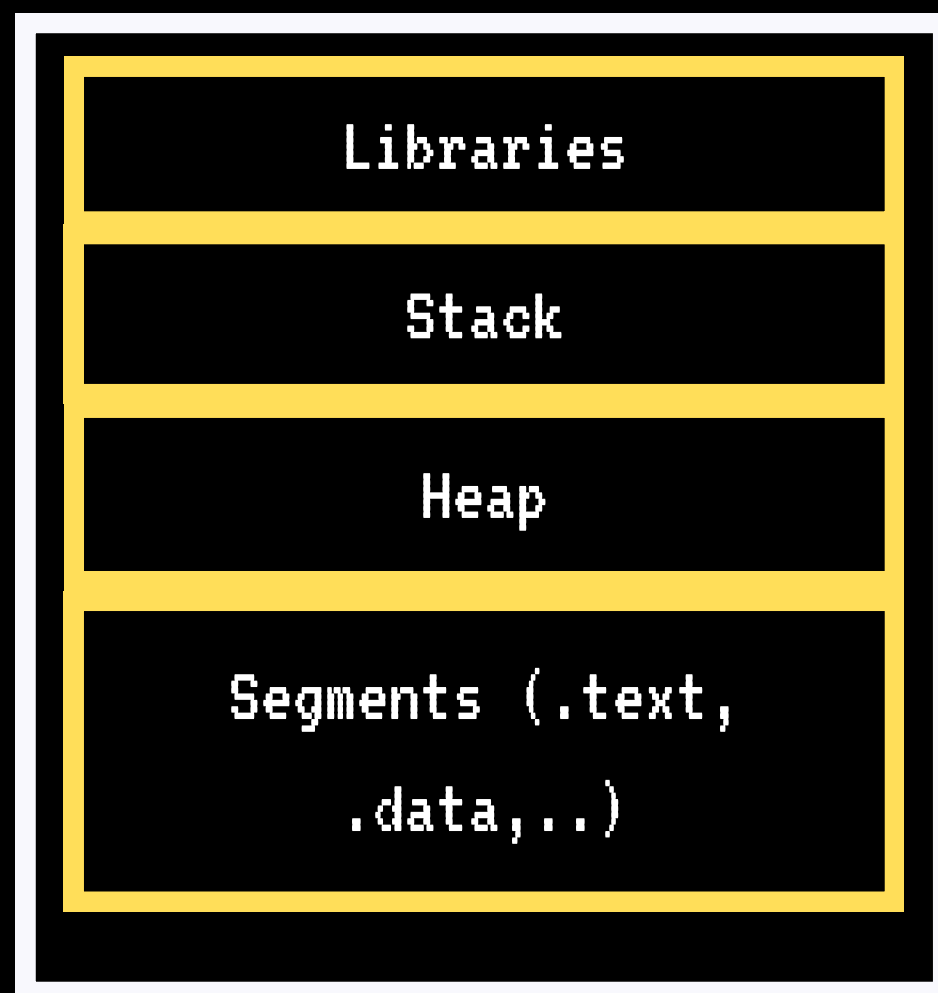
- Stack is non-executable
- Heap is non-executable

* Memory permissions can be changed with syscall mprotect()

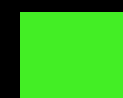
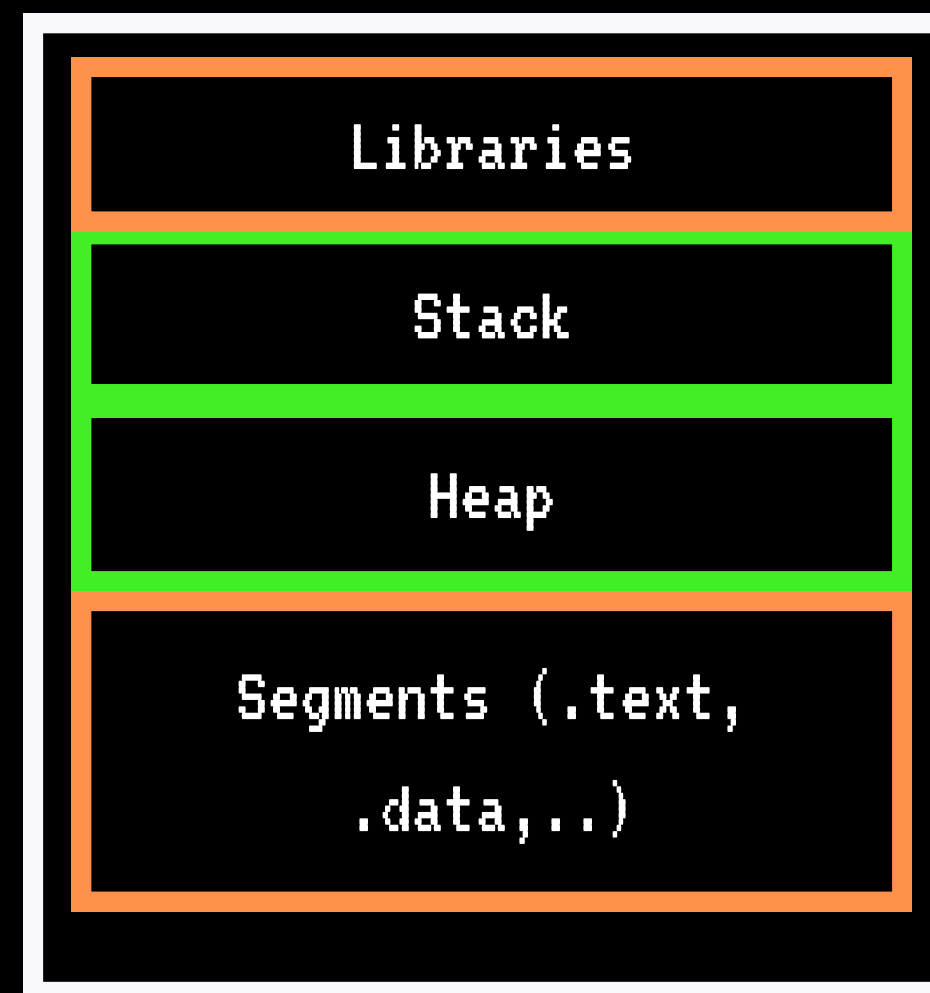
Mitigation - DEP/NX

W^X: Memory is either writable or executable but not both

Before NX bit:



After NX bit:



Writable



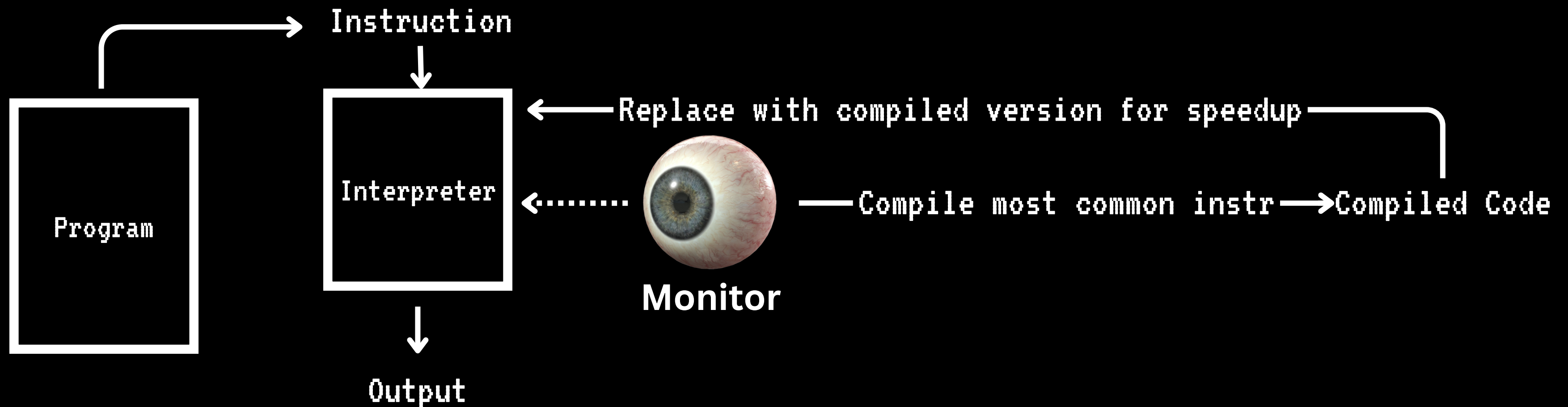
Executable



Write and Executable

NX Bypass - JIT Spraying

JIT (Just In Time Compilation): Mechanism that compiles most commonly executed code while program is running for faster execution



JIT regions are an **exception to W^X** because they must be both executable and writable

NX Bypass - JIT Spraying

JIT Spraying:

1. Make constants in code that will be JITed
2. JIT engine will compile code into memory that is executable
3. Use vulnerability to redirect execution into the JIT region

```
48 ba 48 31 c0 b0 01 0f 50 00    movabs rdx,0x500f01b0c03148
```

JITed assembly

```
let v = 0x050f001b0c03148
```

Evil Code
(Javascript)

```
48 31 c0    xor    rax,rax
b0 01       mov    al,0x1
0f 05       syscall
```

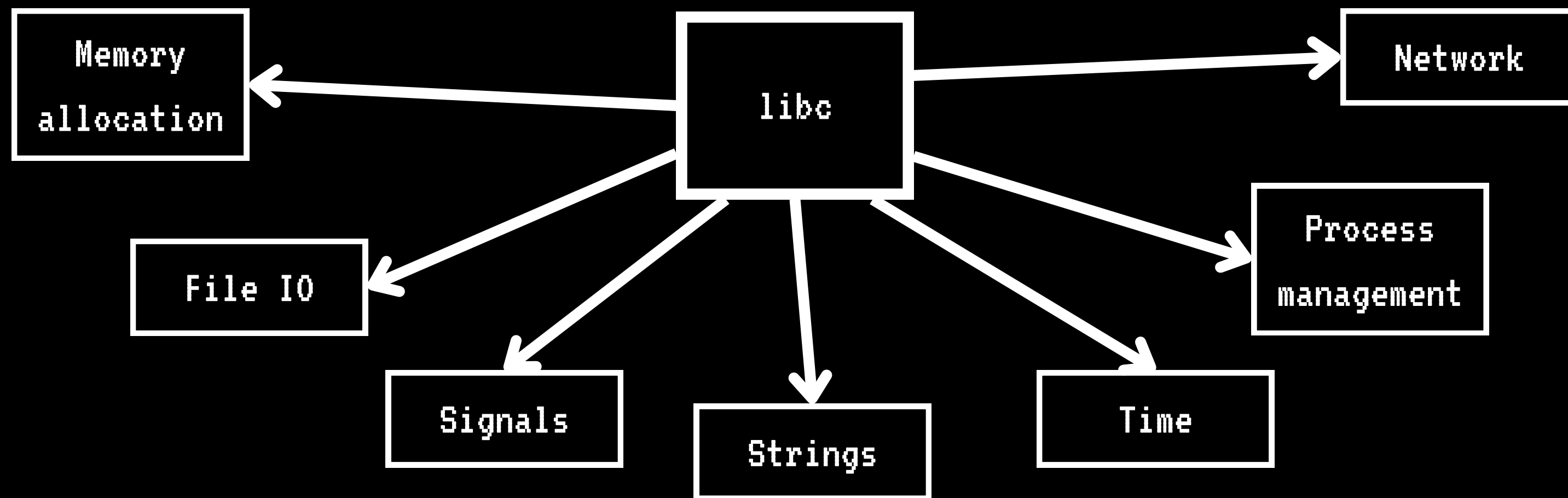
* This is
syscall for
exit()

Partial Portion

Return Oriented Programming

Libc

Libc is a giant shared library file full of C standard library functions



Located at `/lib/libc.so.6` or `/lib64/libc.so.6`

Function finder: <https://libc.blukat.me/>

Return Oriented Programming

Main motivation is to bypass DEP/NX by using libc functions that are allowed to be executed

We locate **ROP Gadgets** which are segments of code present in library or code sections that end in a ret

```
pop rax  
ret
```

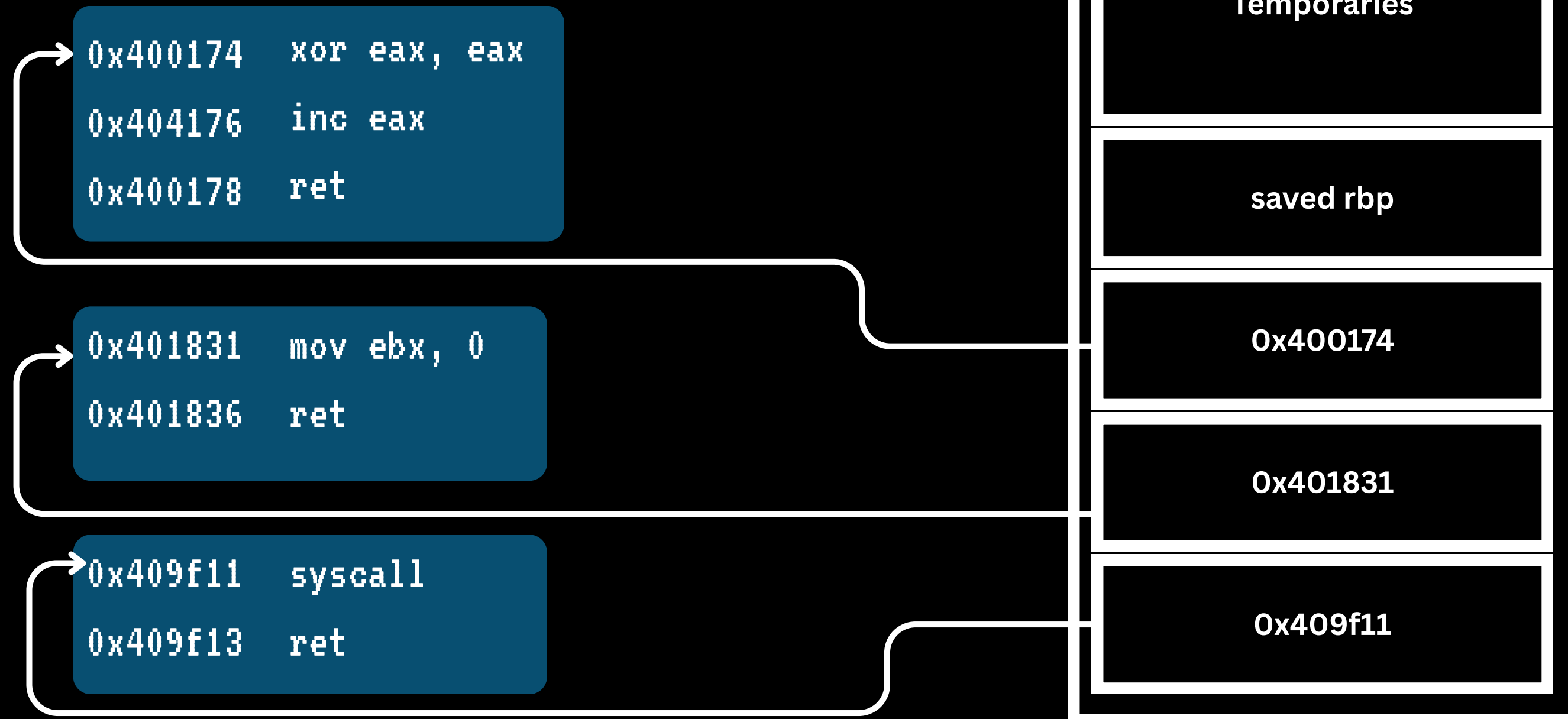
```
add rsp, 0x40  
ret
```

```
pop r12  
pop rdi  
pop rsi  
ret
```

ROP Chains

ROP chain: a collection of ROP Gadgets on the stack that return to each other

* This chain triggers syscall exit()



ROP Example

```
#include <stdlib.h>
#include <stdbool.h>
#include <stdio.h>
bool canopen = false;
void setopen(){
    canopen = true;
}
int win(int key){
    if (canopen && key == 1337) system("/bin/sh");
}
int main(){
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    char buff[100];
    printf("write your post here: \n");
    gets(buff);
    printf("your post: %s\n", buff);
    return 0;
}
```

Follow along with files here:

<https://github.com/UofTCTF/intro-binexp-workshop-challenges>



Finding Gadgets

You can use ROPgadget to find gadgets in a program

```
└─(21:41:17)─> ROPgadget --binary myprog | grep rsp
0x00000000000001016 : add rsp, 8 ; ret
0x00000000000001227 : cli ; sub rsp, 8 ; add rsp, 8 ; ret
0x00000000000001011 : sal byte ptr [rdx + rax - 1], 0xd0 ; add rsp, 8 ; ret
0x00000000000001229 : sub esp, 8 ; add rsp, 8 ; ret
0x00000000000001228 : sub rsp, 8 ; add rsp, 8 ; ret
```

ROPGadget can also generate a ROP chain for you

```
ROPGadget --binary ./someprogram --ropchain
```

<https://github.com/JonathanSalwan/ROPgadget>

ROP - Stack Pivoting

Using a ROP gadgets to change rsp

```
pop rsp  
ret
```

```
pop rdx  
mov rdx, 20  
xchg rdx, rsp  
ret
```

```
leave  
ret
```

Oftentimes good if your stack is too small

Magic Gadgets

These are single gadgets that can be used to win a challenge:

- Open a shell
- Read a file
- Changing permissions

You can find these magic gadgets with OneGadget:

https://github.com/david942j/one_gadget

Mitigation - Control Flow Integrity

Modern processors use Control Enforcement Technology (CET) to prevent indirect jumps by setting the first instruction of a program to **endbr64** marking the start of a branch

```
| <.init>:
f3 0f 1e fa          endbr64
48 83 ec 08          sub    rsp,0x8
48 8b 05 c1 2f 00 00  mov    rax,QWORD PTR [rip+0x2fc1]
48 85 c0             test   rax,rax
74 02               je     1016 <strlen@plt-0x1a>
ff d0               call   rax
48 83 c4 08          add    rsp,0x8
c3                  ret
```

7

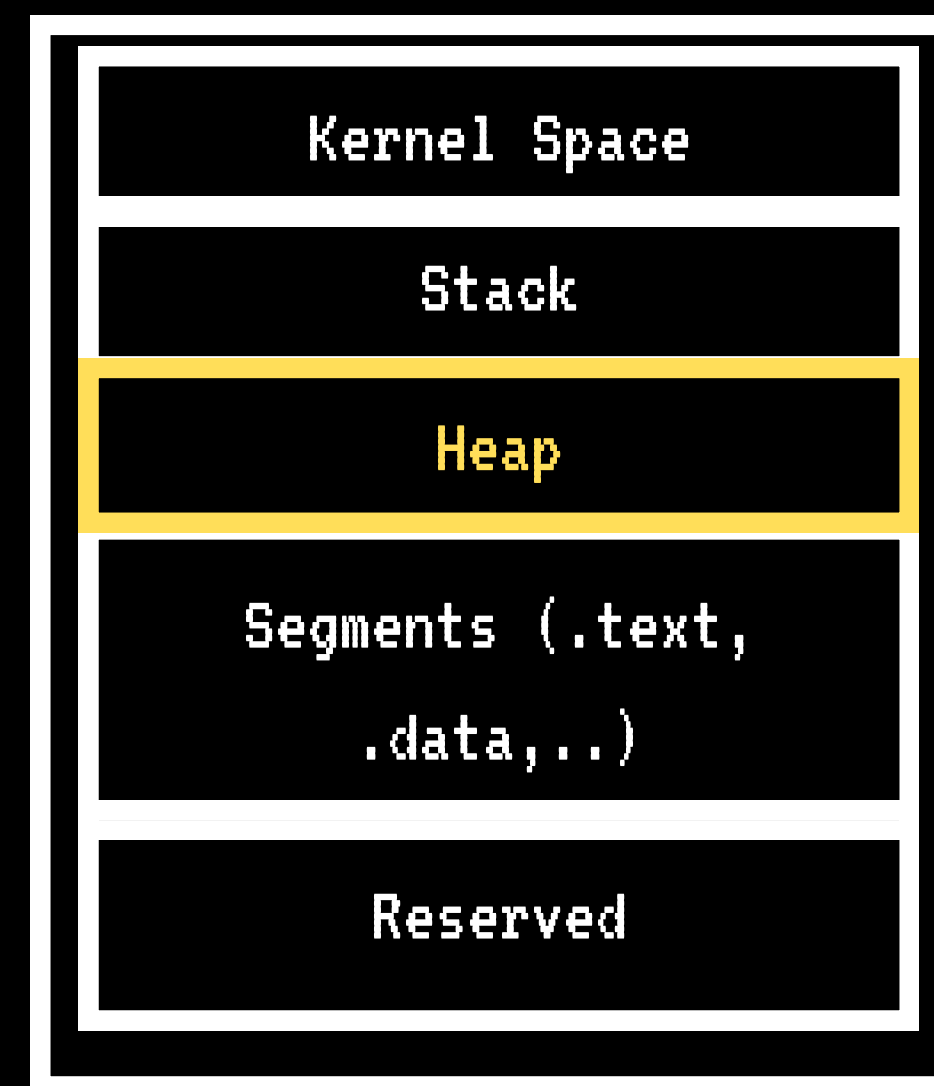
Any jump that does not enter a **endbr64** will raise exceptions

Heap

Heap

The heap is the part of program memory designed for:

- Dynamically allocated data
- Data shared between functions



Virtual Memory

Heap

A portion of the heap can be requested with `malloc()` - it returns a pointer to the memory space

```
void *malloc(size_t size)
```

A portion of the heap can be cleared with `free()`

```
void free(void *p)
```

Developer Rules

Do not read or write to a pointer returned by malloc² after that pointer has been passed back to free.
-----> Can lead to use after free vulnerabilities.

Do not use or leak uninitialized information in a heap allocation.¹
-----> Can lead to information leaks or uninitialized data vulnerabilities.

Do not read or write bytes after the end of an allocation.
-----> Can lead to heap overflow and read beyond bounds vulnerabilities.

Do not pass a pointer that originated from malloc² to free more than once.
-----> Can lead to double free vulnerabilities.

Do not read or write bytes before the beginning of an allocation.
-----> Can lead to heap underflow vulnerabilities.

Do not pass a pointer that did not originate from malloc² to free.³
-----> Can lead to invalid free vulnerabilities.

Do not use a pointer returned by malloc² before checking if the function returned NULL.
-----> Can lead to null-dereference bugs and occasionally arbitrary write vulnerabilities.

¹ Except for calloc, which explicitly initializes the allocation by zeroing it.

² Or malloc-compatible functions including realloc, calloc, and memalign.

³ free(NULL) is allowed and not an invalid-free, but does nothing.

<https://azeria-labs.com/heap-exploitation-part-1-understanding-the-glibc-heap-implementation/>



Heap Allocators

The specific implementations of 'malloc' and 'free'.

Depends on library and architecture.

- glibc malloc (like ptmalloc) (Linux)
- jemalloc (FreeBSD)
- PartitionAlloc (Chrome)
- Segment Heap (Windows)
- kmalloc (Linux kernel)

glibc malloc

Standard heap allocator for Linux

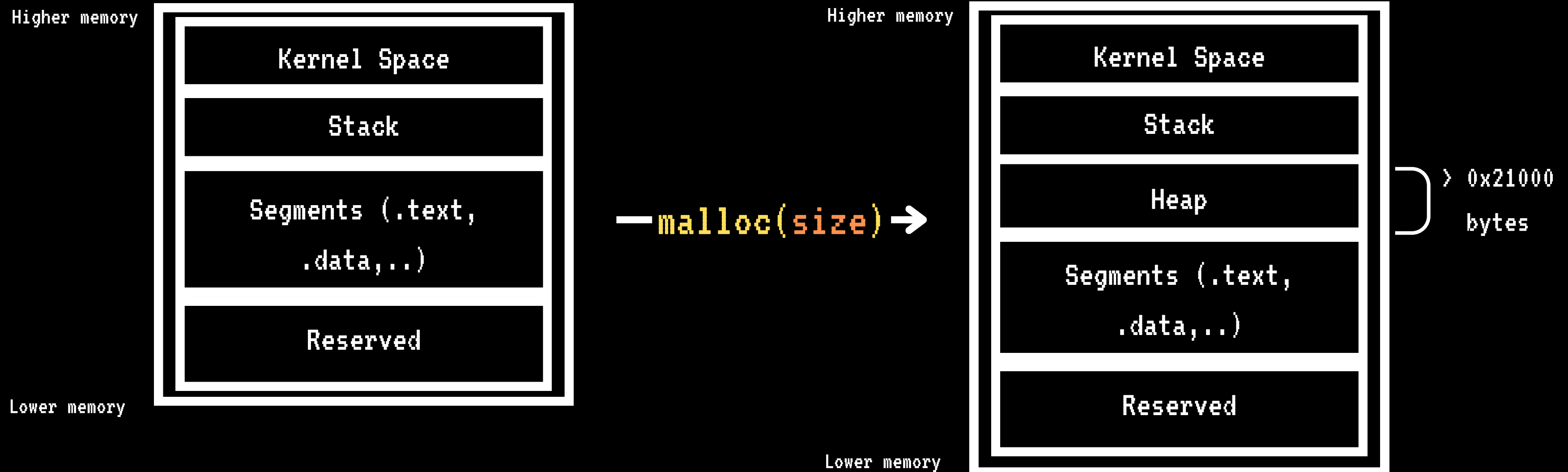
Based off ptmalloc which commonly used because
of multi-thread support



Frequent large changes due to performance & security
updates and patches - existing docs become obsolete

glibc Heap Creation

Heap is only created once first `malloc()` is called



Depending on C macro `DEFAULT_MMAP_THRESHOLD`:

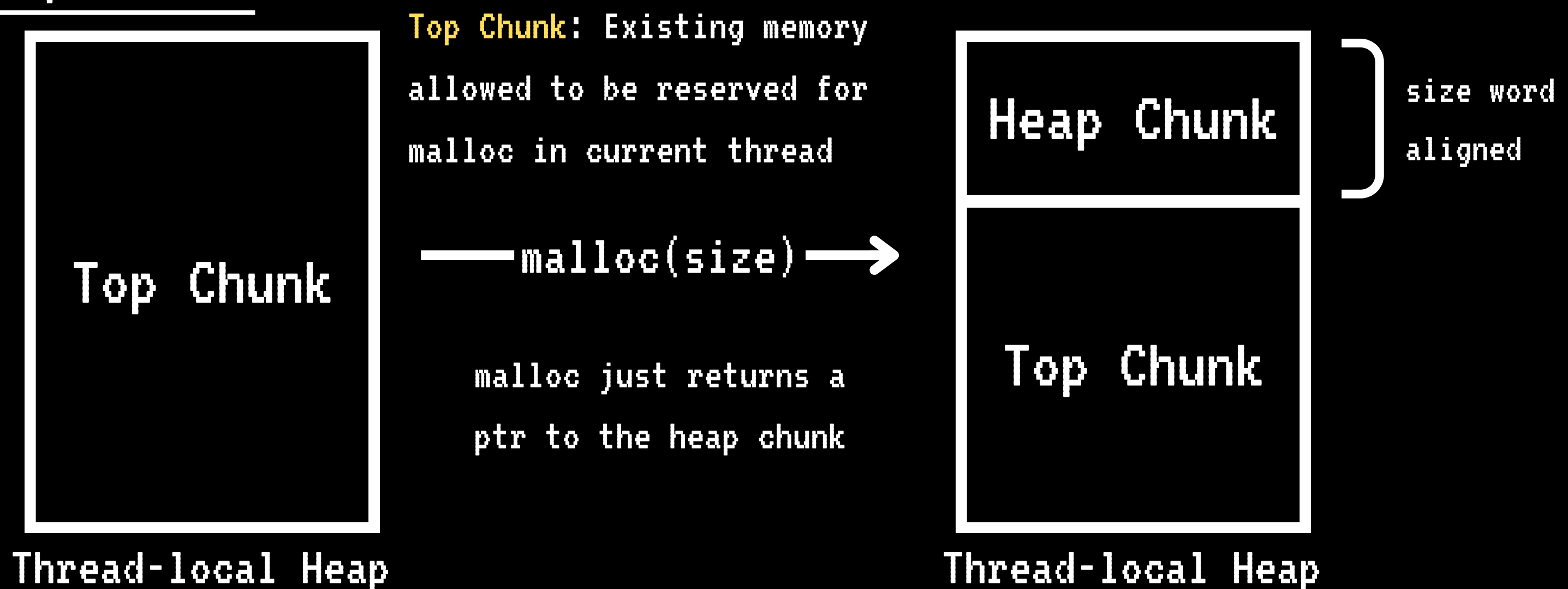
- If requested `malloc(size)` has `size > DEFAULT_MMAP_THRESHOLD`, use `mmap()`
- Else, we use `brk()` to denote heap start and `sbrk()` to denote heap end

Heap - Malloc

Everytime malloc is called:

- If a chunk exists in **Free List**, we use that
- Else, we create a new **Heap Chunk**

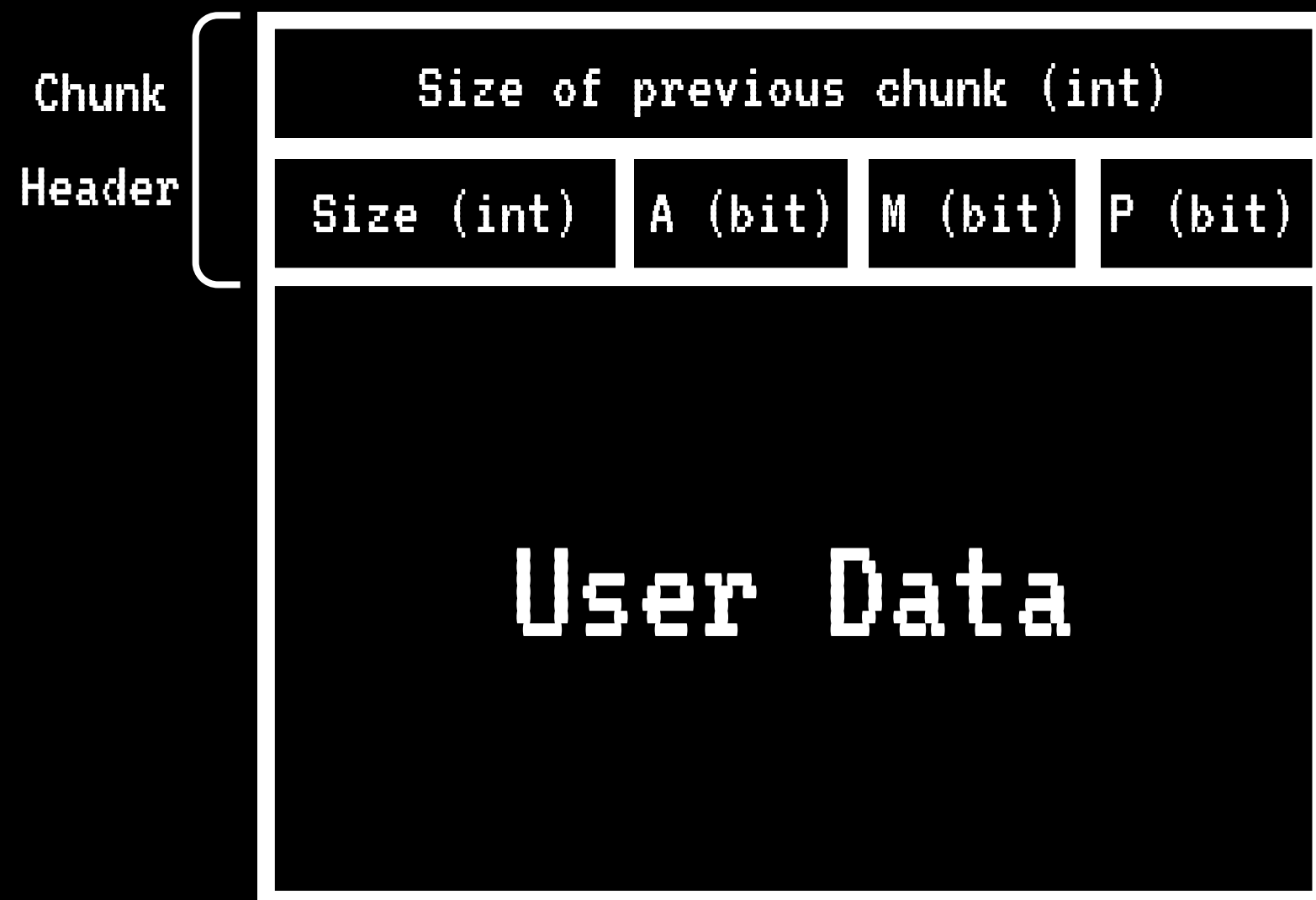
Creating Heap Chunk:



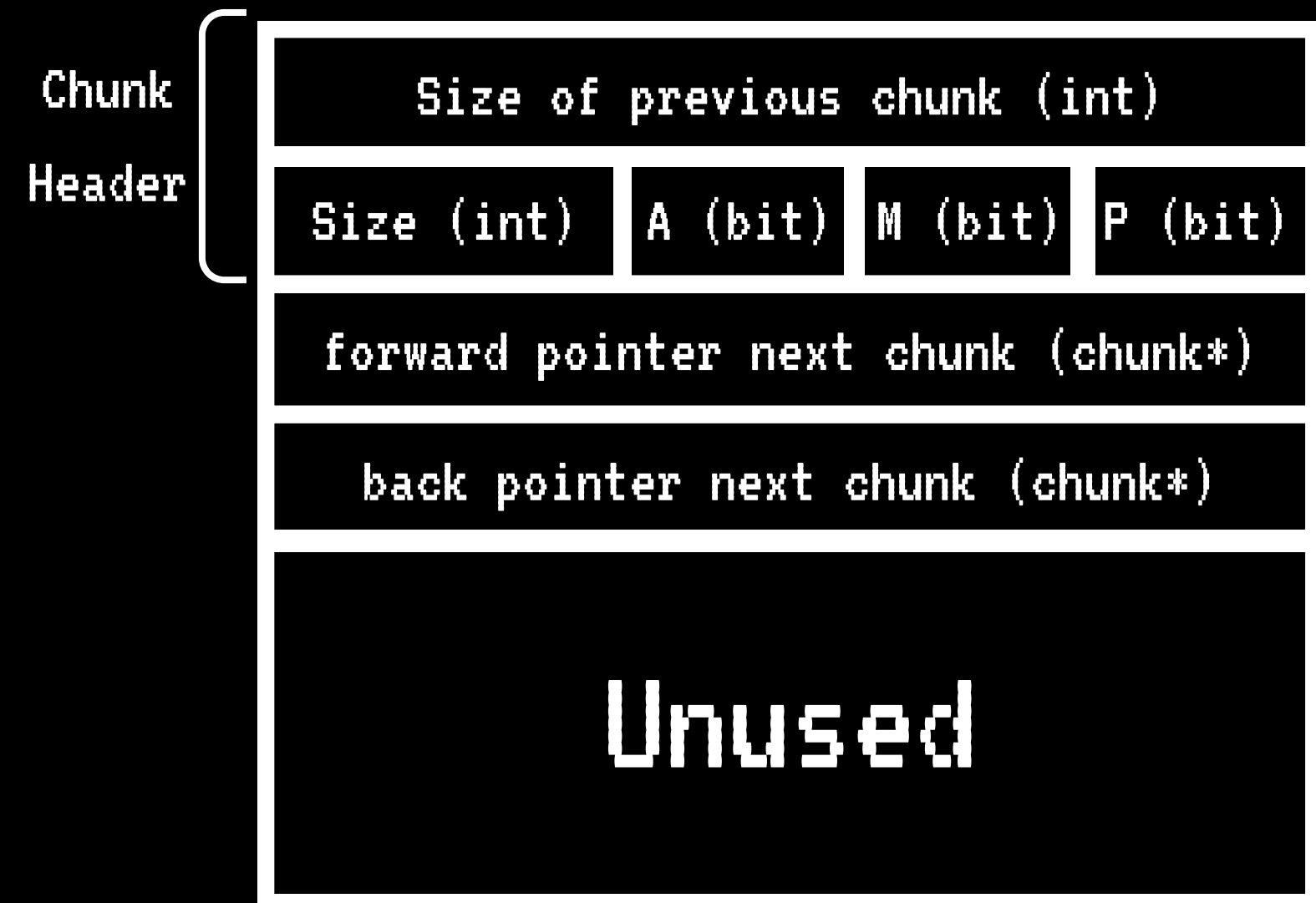
glibc Heap Chunk

Smallest unit of memory in the heap. Contains metadata and actual data

Most importantly, can either be allocated or free



Allocated



Freed

glibc Heap Chunk

If your curious, here is the actual struct:

```
struct malloc_chunk {
    size_t mchunk_prev_size; /* Size of previous chunk, if it is free. */
    size_t mchunk_size;      /* Size in bytes, including overhead. */
    struct malloc_chunk* fd; /* double links used only if this chunk is free.*/
    struct malloc_chunk* bk; /* Only used for large blocks: pointer to next larger size. */
    struct malloc_chunk* fd_nextsize; /* double links -- used only if this chunk is free. */
    struct malloc_chunk* bk_nextsize;
};

typedef struct malloc_chunk* mchunkptr;
```

Heap - Free

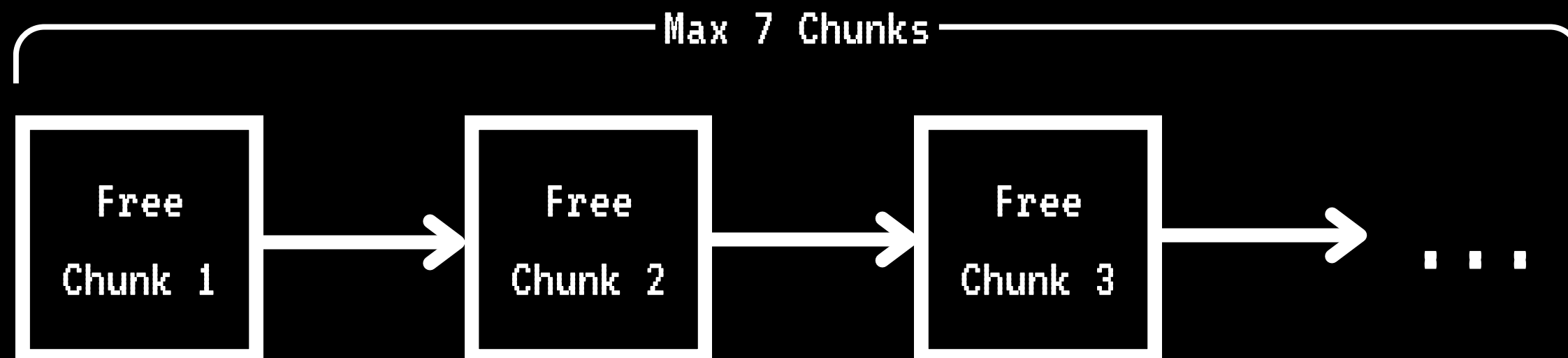
Everytime free() is called:

- We mark the chunk as free, then move it into a **Free List/Bin**
- The **Bin** it goes to depends on the size of a chunk and the capacity of the bin

Teache Bins

Bin for small allocations:

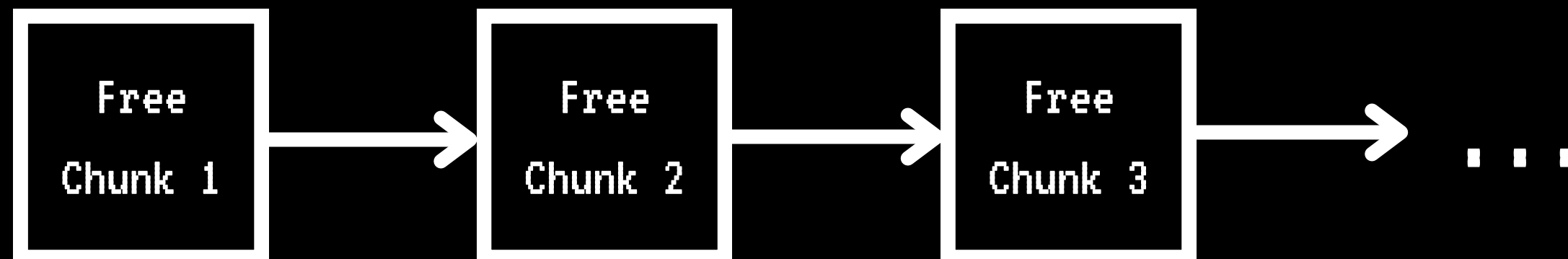
- 0x20 <= Chunk size <= 0x410
 - Max of 7 chunks
 - Singly linked list (only forward ptrs)
- * First malloc() with teache size also sets up teache manager chunk



Fast Bins

Bin for small allocations:

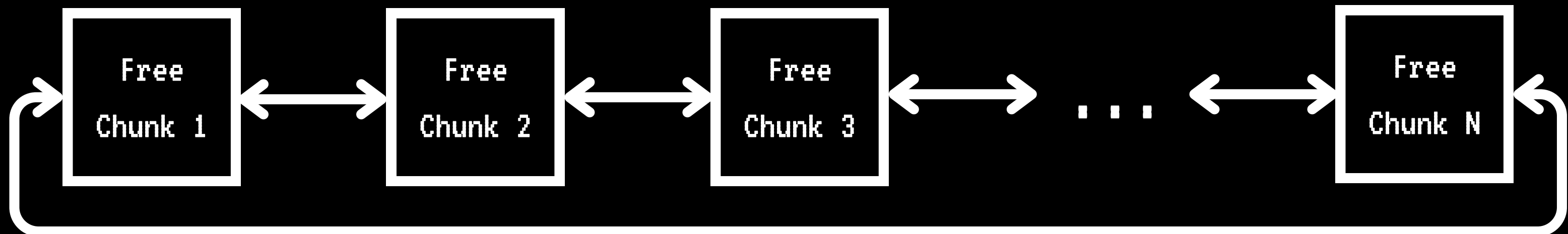
- Used only after we exhaust tcache bins
- $0x20 \leq \text{Chunk size} \leq 0xb0$
- No maximum
- Singly linked list (only forward ptrs)



Unsorted Bins

Bin for larger allocations:

- Chunk size $> 0xb0$
- No maximum
- Doubly circular linked list



Heap Allocation Walkthrough

```
#include <stdlib.h>
```

```
int main() { ← instr pointer  
    int *a = (int*)malloc(0x20);  
    int *b = (int*)malloc(0x420);  
    int *c = (int*)malloc(0x20);  
    free(a);  
    free(b);  
    free(c);  
    int *d = (int*)malloc(0x20);  
    return 0;  
}
```

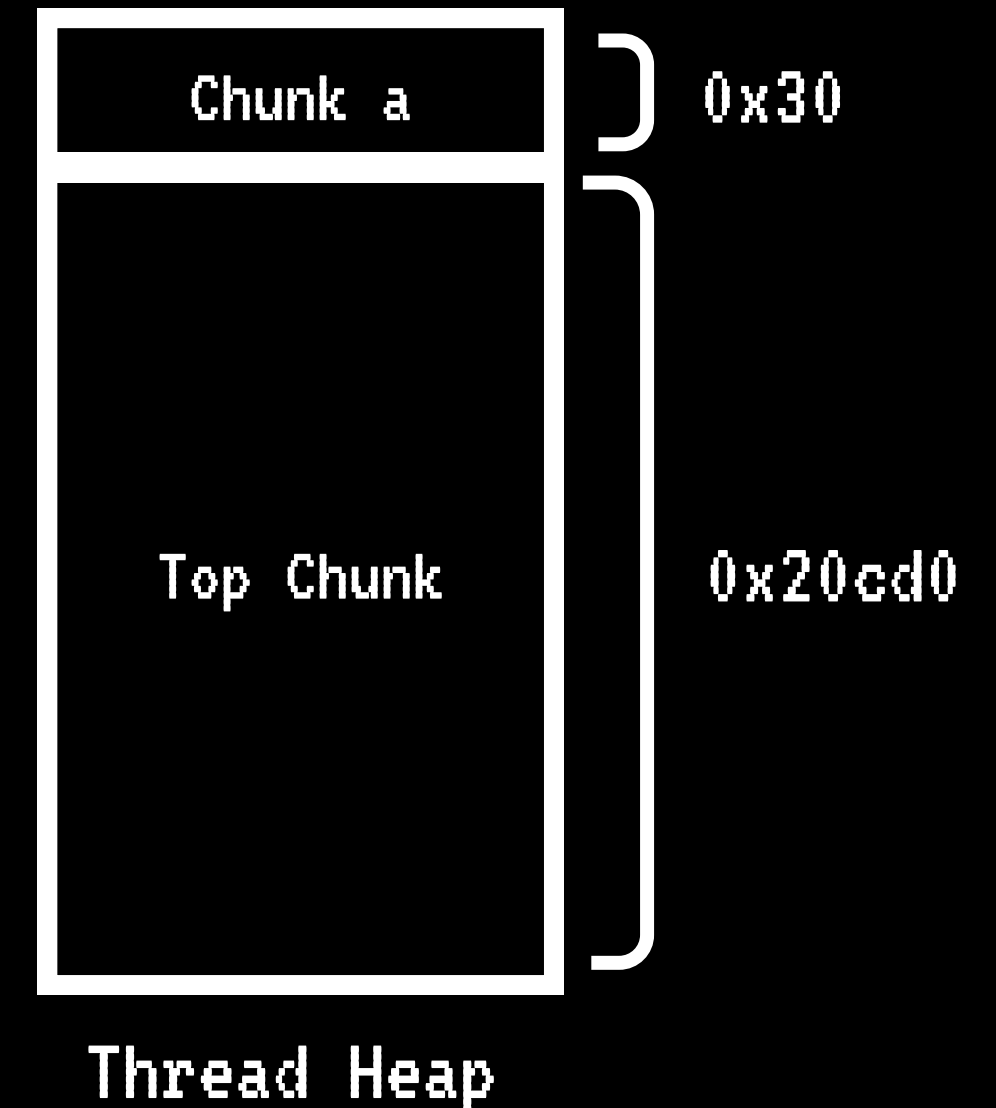
No Heap yet

No Free List yet

Heap Allocation Walkthrough

```
#include <stdlib.h>
int main() {
    int *a = (int*)malloc(0x20); ← instr pointer
    int *b = (int*)malloc(0x420);
    int *c = (int*)malloc(0x20);
    free(a);
    free(b);
    free(c);
    int *d = (int*)malloc(0x20);
    return 0;
}
```

No Free List yet

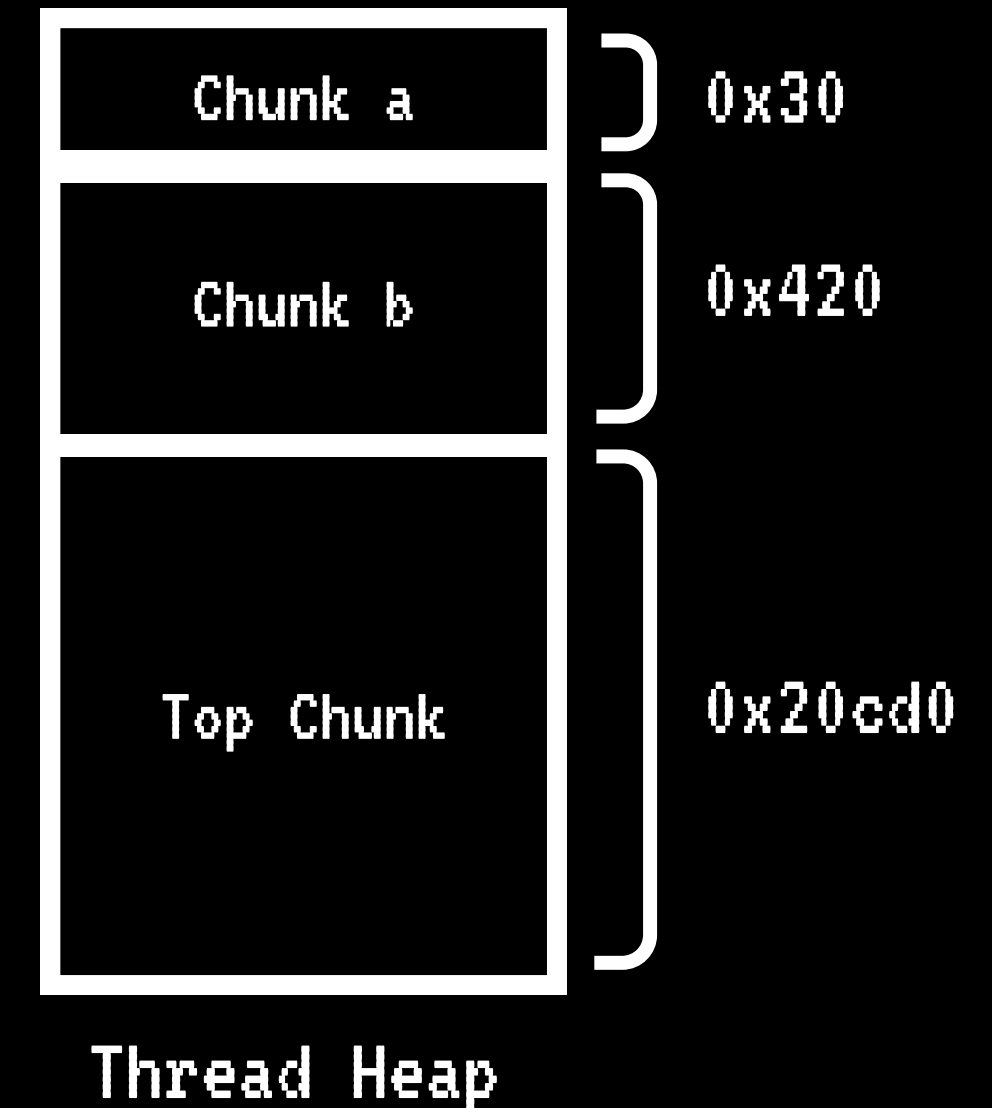


Heap setup, and top chunk split to allocate for first malloc()

Heap Allocation Walkthrough

```
#include <stdlib.h>
int main() {
    int *a = (int*)malloc(0x20);
    int *b = (int*)malloc(0x420); ←instr pointer
    int *c = (int*)malloc(0x20);
    free(a);
    free(b);
    free(c);
    int *d = (int*)malloc(0x20);
    return 0;
}
```

No Free List yet

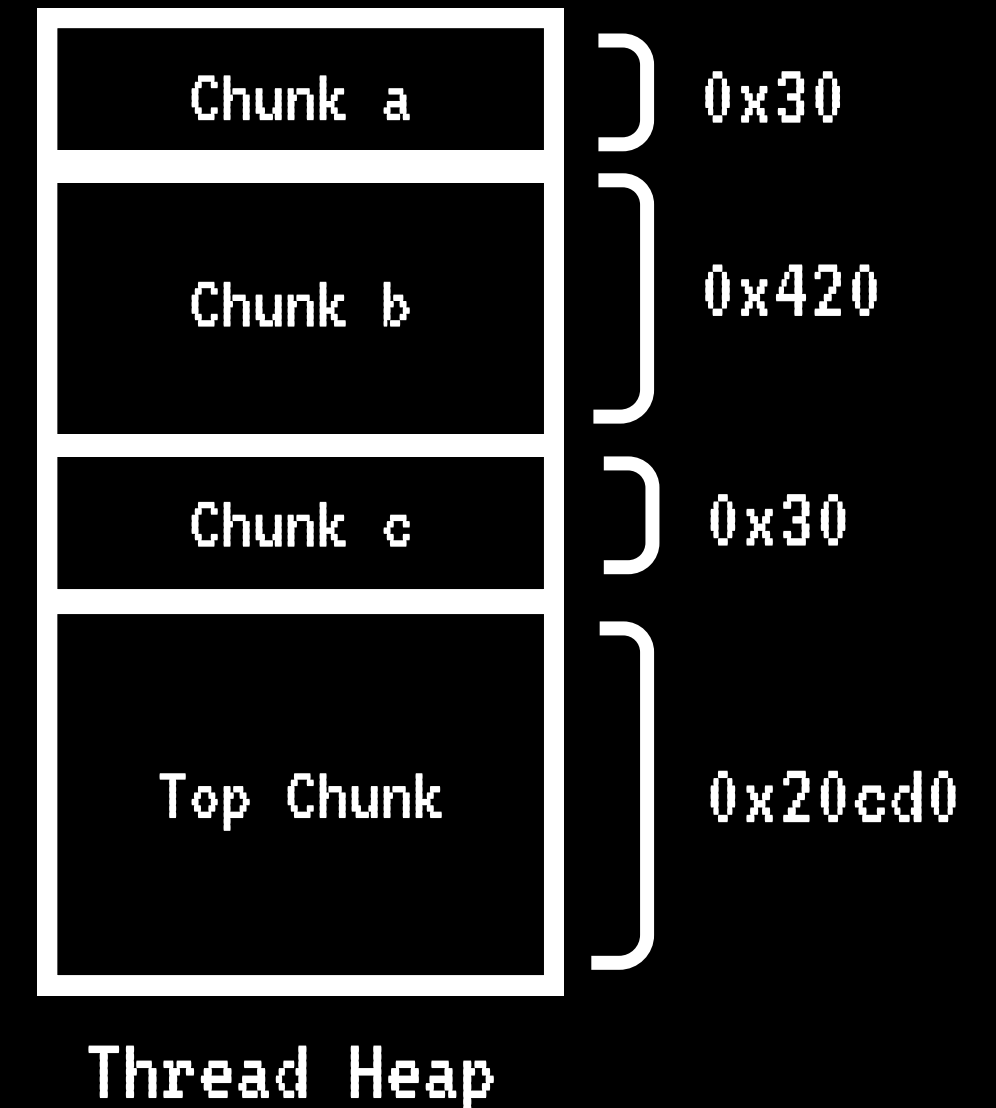


top chunk split for second malloc()

Heap Allocation Walkthrough

```
#include <stdlib.h>
int main() {
    int *a = (int*)malloc(0x20);
    int *b = (int*)malloc(0x420);
    int *c = (int*)malloc(0x20); ← instr pointer
    free(a);
    free(b);
    free(c);
    int *d = (int*)malloc(0x20);
    return 0;
}
```

No Free List yet

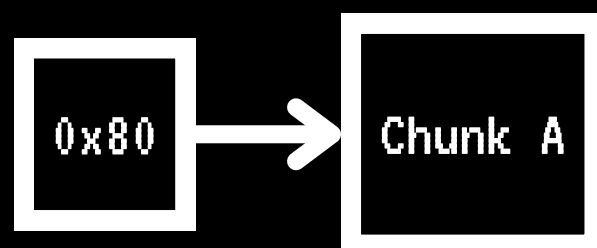


top chunk split for third malloc()

Heap Allocation Walkthrough

```
#include <stdlib.h>
int main() {
    int *a = (int*)malloc(0x20);
    int *b = (int*)malloc(0x420);
    int *c = (int*)malloc(0x20);
    free(a);
    free(b);
    free(c);
    int *d = (int*)malloc(0x20);
    return 0;
}
```

instr pointer



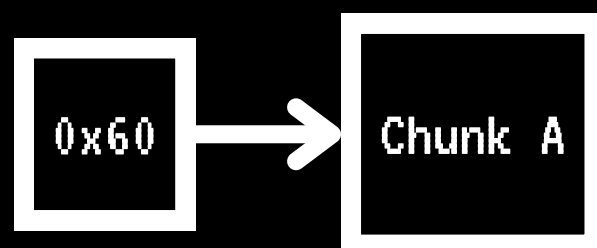
Teache Bin

Free chunk, add to teache list as size is teache size

Heap Allocation Walkthrough

```
#include <stdlib.h>
int main() {
    int *a = (int*)malloc(0x20);
    int *b = (int*)malloc(0x420);
    int *c = (int*)malloc(0x20);
    free(a);
    free(b);
    free(c);
    int *d = (int*)malloc(0x20);
    return 0;
}
```

instr pointer



Teache Bin



Unsorted Bin

Free chunk, add to unsorted bin as size too big

Heap Allocation Walkthrough

```
#include <stdlib.h>
int main() {
    int *a = (int*)malloc(0x20);
    int *b = (int*)malloc(0x420);
    int *c = (int*)malloc(0x20);
    free(a);
    free(b);
    free(c);
    int *d = (int*)malloc(0x20);
    return 0;
}
```

instr pointer



Teache Bin

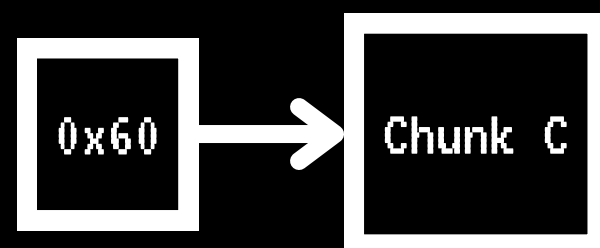
Unsorted Bin

Free chunk, add to teache bin as size is teache size



Heap Allocation Walkthrough

```
#include <stdlib.h>
int main() {
    int *a = (int*)malloc(0x20);
    int *b = (int*)malloc(0x420);
    int *c = (int*)malloc(0x20);
    free(a);
    free(b);
    free(c);
    int *d = (int*)malloc(0x20); ← instr pointer
    return 0;
}
```



Teache Bin



Unsorted Bin

new allocate, take from free list



Use After Free

Vulnerability that exists when program uses
data that was previously freed

The same heap chunk can be re-used for subsequent mallocs

```
int main(){  
    int* a = (int*)malloc(20);  
    printf("%p\n", a);  
    free(a);  
    int* b = (int*)malloc(20);  
    printf("%p\n", b);  
    return 0;  
}
```

```
└─(21:39:16)─> ./a.out  
0x560c713ea310  
0x560c713ea310
```

It is programmers job to set `ptr = NULL` after a `free()` call

Use After Free Example

Lets try and exploit this program:

```
int win(){
    int* winvar = (int*)malloc(0x20);
    if (winvar[4] == 1337) system("/bin/sh");
}
int main(){
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    int* curr_buffer = NULL;
    while (true){
        printf("options: malloc (M), free (F), try win (W) \n");
        char c;
        scanf(" %c", &c);
        switch (c){
            case 'M':
                if (curr_buffer == NULL){
                    curr_buffer = malloc(0x20);
                    printf("data for allocated memory (0x20 bytes):");
                    scanf("%d", curr_buffer + 4);
                }break;
            case 'F':
                if (curr_buffer != NULL) free(curr_buffer);break;
            case 'W':
                win();break;
            default:
                break;
        }
    }
    return 0;
}
```

<https://github.com/UofTCTF/intro-binexp-workshop>

Double Free

A vulnerability where calling on the same chunk can add it to the free list twice (Only work for fast bins)

```
int main(){  
    // ... tcache bin fullled up  
    int *a = (int*)malloc(0x22);  
    free(a);  
    free(a);  
}
```



Fast Bin

Allows the same chunk to be given to two different future malloc() calls

Staggered Double Free

A method to setup double free most libraries dont catch
(works with tcache)

```
int *a = (int*)malloc(0x22);  
free(a); // a ptr not in bin  
int *b = (int*)malloc(0x22);  
// staggered free  
free(b); // UAF to get same chunk  
free(a); // a's ptr not in bin so ok
```

<https://www.youtube.com/watch?v=6ZxmrJEh4Z0>

Heap Overflow

Similar to buffer overflow, but for heap.

```
int main(int argc, char **argv){
    char *a = (char*)malloc(8);
    char *b = (char*)malloc(8);
    printf("b's value: %s\n", b);
    strcpy(a, argv[1]);
    printf("b's value: %s\n", b);
}
```

```
└─(01:28:25)─> ./a.out aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
b's value:
b's value: aa
```

Certain functions like `strcpy()` do not check size

* C++ VTable pointer can also lead to heap buffer overflow

Tcache Poisoning

Combination of UAF and heap overflow

1. Allocate two chunks to heap

2. Free two chunks to get them into tcache bin



Tcache Bin

3. UAF then Heap overflow to overwrite second freed chunk's data



Tcache Bin

4. Malloc again to get the second bin's data

Mixed Bag

Format Strings

Using printf to disclose elements on the stack

- `%x` reads hex bytes off the stack
- `%d` reads decimal bytes off the stack
- `%p` reads pointers off the stack
- `%n` allows users to overwrite memory

```
(kali@kali)-[~/ctf/pico/binary_exploitation/flag_leak]
$ ./vuln
Tell me a story and then I'll tell you one >> %x.%x.%x.%x.%x.%x.%x.%x.%x.%x.%x.%x.
Here's a story -
ffe14c60.f7d6ca6c.8049346.252e7825.78252e78.2e78252e.252e7825.78252e78.2e78252e.252e7825.7
8252e78.2e78252e.
./vuln: flag[BUFSIZ] = 0x0000000000000000000000000000000000000000000000000000000000000000
```


Integer Overflow

Vulnerability wherein integers exceed their maximum allowable size making them wrap around.

- Unsigned integers wrap to zero
- Signed integers wrap to large negative number

```
int main(){
    unsigned int a = 4294967295;
    printf("%u\n", a); // 4294967295
    printf("%u\n", a+1); // 0
    return 0;
}
```

Array Indexing

Array sizes are not enforced, so user input can override access out of buffer memory and read/write stack variables

```
int main(){  
    int array[10];  
    int index;  
    scanf("%d", &index);  
    int value = array[index];  
    return 0;  
}
```

Resources

PwnCollege (GOAT):

- <https://pwn.college/>

Wargames:

- <https://pwnable.kr/>
- <https://pwnable.tw/>
- <https://ropemporium.com>

Resources

Program Memory:

- <https://storage.googleapis.com/google-code-archive-downloads/v2/code.google.com/corkami/elf101.pdf>
- <https://www.youtube.com/watch?v=msUPyCtJPSw>
- <https://elswix.com/articles/6/PLT-and-GOT.html>

Hacktricks:

- <https://angelica.gitbook.io/hacktricks/binary-exploitation/common-exploiting-problems>
- <https://ir0nstone.gitbook.io/notes/binexp/stack/stack-pivoting>
- <https://book.hacktricks.wiki/en/index.html>

Resources

Heap:

- <https://azeria-labs.com/heap-exploitation-part-1-understanding-the-glibc-heap-implementation/>
- <https://sigpwny.com/meetings/general/2025-03-09/>
- <https://www.youtube.com/watch?v=Svm5V41eEho>

Misc:

- <https://www.youtube.com/@yeongjin-jang>

Resources

Libc:

- <https://libc.blukat.me/>
- <https://libc.rip/>
- <https://ftp.gnu.org/gnu/libc/>
- <https://elixir.bootlin.com/linux/>
- <https://naup.mygo.tw/2024/09/23/Pwnable-realloc/>

Resources

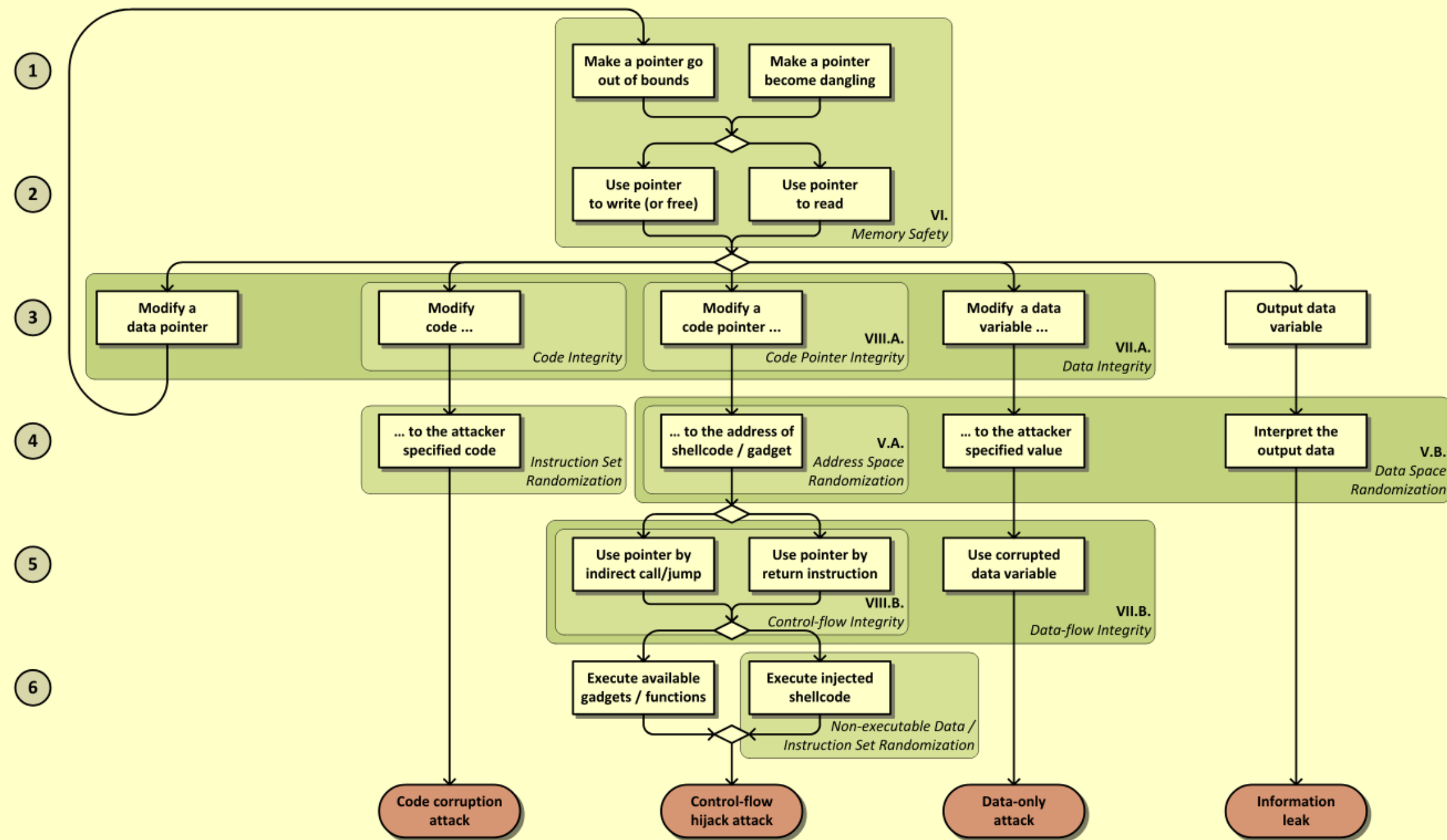


Figure 1. Attack model demonstrating four exploit types and policies mitigating the attacks in different stages

<https://oaklandsok.github.io/papers/szekeres2013.pdf>